

O-RAN, edge data centre, IoT security

What Held-Out Accuracy Predicts About Deploying Intrusion Detection in O-RAN

ARSHAD ALI¹ AND ADEEL AHMAD¹

¹Faculty of Computer and Information Systems, Islamic University of Madinah, Madinah, Saudi Arabia

ABSTRACT Edge data centres in Open Radio Access Network (5G and O-RAN) deployments receive telemetry from very large numbers of IoT devices, and compromised IoT devices are a documented source of control-plane signalling overload. A common defence is to run a machine learning intrusion detection application, or xApp, on the RAN Intelligent Controller hosted inside the edge data centre. Published work in this area routinely reports detection accuracy above **intrusion detectors are often reported to exceed 99%**, measured on a held-out **accuracy on a random split of a single dataset**. This paper asks what that number is worth, and answers that the evaluation protocol, rather than the choice of architecture, determines almost all of it.

We evaluate eight architectures, including two trivial baselines, on a single pipeline across a ladder of four protocols of increasing realism: a random split, a run-disjoint split, a time-disjoint split spanning days of real capture time, and an independently collected deployment. Each step is closer to what a deployment experiences and each costs more than the choice of architecture does. **corpus. We examine what such results predict about deployment, using NetsLab-5GORAN-IDD, 5G-NIDD, six architectures, two input-blind baselines, and increasingly strict protocols.** A random split **inflates raises radio-layer macro- F_1 by** to , and the inflation tracks model flexibility while the trivial baselines gain nothing, identifying memorisation as the mechanism. A time-disjoint split costs a further and nearly triples the alert burden. Under cross-deployment transfer, all six non-trivial architectures degrade significantly; the best clears a stratified coin flip on the target by **0.13 (95% interval -0.01 to 0.27, significant only for two sequence models), and a nearest-neighbor lookup gains 0.15, consistent with the recognition of capture sessions. The false positive rate on a held-out benign session ranges from 0.00 to 0.98. In 5G-NIDD, 59% of the benign flows duplicate flood records labeled as attacks, and their addresses identify them as the flood of a second attacker. These copies limit balanced accuracy to 0.766, and the architecture with the strongest held-out source score falls beneath that floor, with published 99.9% accuracy depends on two record-position fields, without which it decreases to 76.9%. On flows with consistent labels, transferred detectors reach a balanced accuracy of** . Across the eight, in-distribution rank does not predict transfer rank.

Operationally, corpus precision of approximately corresponds to a pooled deployment precision between and at a declared base rate, a collapse by a factor of that is stable across four orders of magnitude of assumed prevalence. We show that this quantity is routinely mis-estimated: averaging precision across evaluation folds rather than pooling confusion counts inflates the apparent spread between architectures from to on our data, and we demonstrate the error on our own previously reported figures. We further show that calibration cannot repair the operating point, because every standard calibration method is a monotone transform and therefore cannot change the achievable operating points at all; threshold saturation is a discriminability limit, not a calibration failure. Finally, a stage decomposition of decision latency identifies feature extraction rather than inference as the dominant cost, and it remains dominant after an order-of-magnitude optimisation of the extractor.

Our latency measurements are emulated and support no **0.61 to 0.78, below the in-target reference, and re-extracting the target with the exporter of the source does not close the gap. A third corpus shows the same pattern, and no detector passes a deployability test that the published pipeline passes on its own split. At a prevalence of 0.002, operational precision stays at or below 0.063 at every threshold that keeps recall above 10%. Latency is the one criterion met. Through a FlexRIC near-real-time conformance claim, and our two corpora were produced by different flow exporters, so every reported transfer gap is an upper bound spanning both a change of deployment and a change of tooling. We conclude with a set of reporting requirements that future O-RAN security papers should meet before claiming deployability RIC with an emulated E2 node, a decision and its control message take at most 5.01 ms at the 99th percentile. We close with reporting requirements for deployability claims.**

INDEX TERMS Dataset shift, evaluation methodology, false positive rate, intrusion detection, xApp, RAN Intelligent Controller, model generalisation, false positive rate label quality, O-RAN, RAN intelligent controller, xApp.

I. INTRODUCTION

Open Radio Access Network (O-RAN) architectures disaggregate the base station into radio, distributed and centralised units connected by , and central units that are connected through open interfaces, and place they introduce a RAN Intelligent Controller (RIC) alongside them to host that hosts third-party control logic [1]–[3]. In most commercial designs applications [1]–[3]. Because the near-real-time RIC is not deployed at the cell site but inside a regional edge data centre that aggregates traffic from tens to hundreds of cells [4], [5]. That data centre therefore sees the union of the control-plane and user-plane telemetry generated by every IoT device attached to those cells operates close to the cells it controls [1], [5], it has access to the telemetry of every device served by those cells. Compromised devices, many of which are IoT devices, which makes it a natural vantage point for detecting compromised devices before their aggregate behaviour degrades the radio network.

The motivating threat is well documented. Large IoT botnets have repeatedly been assembled from devices with weak or default credentials [6], [7], and in cellular deployments the damage is not limited to volumetric flooding of an external target. Bursts of attach, detach and radio resource control transitions from a synchronised population of devices impose a signalling load that scales with the number of state changes rather than with the number of transmitted bytes, and this class of signalling storm has been shown to degrade a cellular network well before any link saturates [8]. Detecting the underlying compromise early, from inside the RAN control loop , is consequently attractive are a well-documented source of volumetric attacks [6], [7]. A detection application (xApp) placed in the RIC control loop is therefore an attractive defensive option. Platforms for developing xApps are available [9], [10], and detectors that operate within or alongside the O-RAN control loop have already been demonstrated [11], [12].

The natural implementation is a detection xApp: a containerised application that subscribes to E2 service model reports, extracts features, classifies traffic, and issues control actions through the same interface [9], [10]. A substantial and growing literature proposes such xApps, and the headline result is remarkably consistent across papers: detection Detectors for 5G and O-RAN traffic are typically evaluated on a random held-out split of a single dataset, and the reported accuracy or F_1 above score commonly exceeds 99% on a held-out split of a single dataset. What is far less consistent is any evidence that the detector would survive contact with a production network.

Three properties determine whether . For 5G-NIDD, one of the two corpora used in this study, the dataset authors report binary accuracies between 99.85% and 99.95% for four of five classifiers [13], and subsequent work reports a weighted F_1 score of up to 99.95% [14]. However, whether such a detector can be deployed depends on three properties that a detector is deployable, and none of them is measured by a held-out split of the training distribution does not measure. The first is cross-deployment generalisation. A model trained on telemetry from one radio configuration, device population and traffic mix must classify correctly in a different one, and network intrusion detection models generalization to a deployment that did not contribute training data, and intrusion detectors are known to transfer poorly between independently collected datasets [15], [16]. The second is alert burden. A the alert burden, since a false positive rate that appears negligible on a class-balanced benchmark becomes an unmanageable alert volume small on a balanced benchmark produces a large volume of

alerts once it is multiplied by production flow rates and discounted by interpreted against a realistic attack base rate, a consequence of the base-rate fallacy identified for intrusion detection more than two decades ago [17], [18]. The third is tail latency . The latency relative to the near-real-time RIC control loop is specified to operate control loop, for which the O-RAN E2 specification requires support for latencies between 10 ms and 1 s [2], and a detector whose tail latency exceeds the budget does not merely respond slowly, it fails to close the loop at all. That specified range spans two orders of magnitude, which means a paper can reach opposite conclusions about the same measurements by choosing where in it to draw the line [19]. A benchmark score thus indicates how well a detector separates the records of one corpus, whereas deployment concerns its behavior on traffic it was not trained on, at a realistic base rate, and within a control loop. In this paper, benchmark results are treated as evidence about deployment only when an experiment supports that interpretation.

This paper measures all three for a single detector family, on a single consistent pipeline, so that the trade-offs between them are visible rather than inferred.

The framing matters, because cross-deployment transfer between 5G intrusion detection corpora has been measured before, including between None of these concerns is new. Axelsson made the base-rate argument [17], and Sommer and Paxson the closed-world critique [18]. Arp et al. catalogued common pitfalls of machine learning in security [20], and Pendlebury et al. formalized temporal and spatial bias [21]. Abraheem and Edhirig measured transfer between the two corpora used here [22]. Our contribution is not we use [22], and Obiuevwi et al. measured inference inside a real near-real-time RIC [23]. The contribution of this paper is neither a new detector nor the observation that transfer degrades performance. It is that the evaluation protocol, not the architecture, dominates every number a reader of this literature is given, and that this can be quantified. Our instead a single controlled evaluation of one detector family in which these effects are measured jointly, with input-blind baselines evaluated on every corpus, so that their relative magnitudes can be compared and the claims that commonly rely on them can be tested. The specific contributions are as follows.

- We define a three-criterion deployment evaluation protocol for O-RAN detection xApps — cross-deployment generalisation, operational alert burden under a realistic base rate, and tail latency against the near-real-time budget — and give a formal deployability predicate that combines them evaluate a ladder of evaluation protocols on a single pipeline with six architectures and two input-blind baselines. A model-free nearest-neighbor probe and two sequence models indicate that the random-split gain arises from the recognition of capture sessions (Section VI-A). A split that fixes category coverage further shows that the radio corpus does not support an estimate of temporal drift and that the false positive rate depends mainly on which benign session is tested (Section III). VI-B).

- We measure a ladder of four evaluation protocols on one corpus, one pipeline and one set of eight architectures, and report the cost of each rung: a random split, a run-disjoint split, a time-disjoint split over days of real elapsed capture time, and an independently collected deployment. Each step is closer to what a deployment experiences and each costs more than the choice of architecture does. We audit the target corpus and show that half of the 5G-NIDD flows lie on records that occur under both labels, that the addresses of the benign copies identify them as the flood of a second attacker, and that a published 99.9% result on this corpus depends on record-position fields. We then evaluate the published pipeline under the same ladder of protocols (Section VI). VI-C).
- We evaluate trivial baselines on the target corpus, not only on the source. This is what makes a transfer number readable. We separate the contributions of prior shift, label conflicts, and transfer failure to the cross-corpus gap by means of a prevalence-invariant metric, a difference-in-differences test against the majority baseline, references measured within the target corpus, and evaluations with and without the conflicting records. We repeat the transfer after re-extracting the target from its raw captures with the exporter of the source, and it changes the conclusion: the architecture with the best held-out sourcescore transfers to below a stratified coin flip, and across eight architectures the rank correlation between in-distribution and transfer performance is not significant (Section VI). on a third corpus collected at a different site with a different exporter (Sections VI-D, VI-E, and VI-F).
- We identify and quantify an estimator error in how operational metrics are computed. Averaging precision across evaluation folds, rather than pooling. We report operational precision and alert volume from pooled confusion counts, in explicit units and with cluster-bootstrap intervals, and we show that averaging precision over evaluation folds inflates the apparent spread difference between architectures by an order of magnitude, and the distortion grows as the number of negatives per fold shrinks — precisely the regime in which the operational questions are interesting. We demonstrate this on our own previously reported figures (Section VI). VI-G).
- We show that calibration cannot repair the operating point, and why it cannot. Every standard calibration method is a monotone transform, so none changes the achievable set of operating points; threshold saturation is a discriminability limit rather than a calibration failure. We identify which calibration methods can alter the achievable operating points, show that none of them improves these operating points in the present setting, and show that label-free estimation of the target prior fails when the class-conditional distributions shift (Section VI). VI-H).
- We decompose decision latency and show that feature extraction,

TABLE 1. Notation and abbreviations.

Symbol	Meaning
$\mathcal{D}_A, \mathcal{D}_B$	source corpus (NetsLab-5GORAN-IDD); target corpus (5G-NIDD)
$\mathcal{D}_C$	third corpus, a second target (5G core datasets of [24])
τ	decision threshold on the attack score $\hat{p}$
$R(\tau), \text{FPR}(\tau)$	recall and false positive rate at τ
π	operational attack prevalence (declared, not measured)
π_{tr}	attack prevalence of the training data
λ_b	benign arrival rate (flows or windows per hour)
$A(\tau)$	false alerts per hour at τ
$\text{PPV}(\tau)$	operational precision at prevalence π
$\Delta_{BA}, \Delta_{F_1}$	source minus target balanced accuracy, macro- F_1
$\bar{W}_1$	mean first-order Wasserstein distance over features
$L, q_p(L)$	decision latency and its p -th quantile
B, ε	latency budget and tolerated violation rate
$\delta, A_{\max}, r_{\min}, \rho$	bounds of the deployability test, Eq. (6)
δ_m	deadline margin in Algorithm 1 (design, not evaluated)
BA	balanced accuracy, the mean of TPR and TNR
LR, DT, RF	logistic regression, decision tree, random forest
XGB, HGB, MLP	XGBoost, histogram gradient boosting, multilayer perceptron
IF, AE, GRU	isolation forest, autoencoder, gated recurrent unit
KPM, RC	E2 service models for measurements and RAN control
UE	user equipment

not inference, is the dominant cost, and that it remains dominant after an order-of-magnitude optimisation of the extractor. Comparison against a published measurement taken inside a real Near-RT RIC [23] indicates that the conformance verdict is a property of the implementation rather than of the architecture on one host for three implementations, including a compiled runtime, and time the full decision loop of the detector as an xApp in a FlexRIC near-real-time RIC with an emulated E2 node (Section VI).

- We provide a concise reporting checklist that we argue should accompany any O-RAN security paper claiming deployability (Section VI-I). Finally, we combine generalization, alert burden, and latency into a deployability test, examine how its verdict depends on its bounds, and derive reporting requirements (Sections VII and X).

What this paper does not do. Our latency measurements are *emulated*: they are taken off-platform, with no RIC in the path, and they support relative ordering and stage decomposition only. We make no **Latency is measured on one host, once without a RIC and once through a FlexRIC near-real-time** conformance claim and we do not report container CPU or memory footprints, because we did not measure them. Section IX states the remaining constraints, of which the most important is that our two **RIC connected to an emulated E2 node. No radio or operator network was involved.** The three corpora were produced by three different flow exporters, so every reported transfer gap is an upper bound spanning both **each cross-corpus gap mixes** a change of deployment and **with** a change of tooling. For $\mathcal{D}_A$ and $\mathcal{D}_B$ we remove this confound by re-extracting $\mathcal{D}_B$ with the exporter of $\mathcal{D}_A$ (Section VI-E). The notation used throughout the paper is summarized in Table 1.

II. BACKGROUND AND RELATED WORK

A. THE NEAR-REAL-TIME RIC AS A SECURITY VANTAGE POINT

We review work on the RIC as a detection platform, on O-RAN security, on datasets and learning

methods for network intrusion detection, and on evaluation practice and dataset shift, from which our methodology draws.

A. THE NEAR-REAL-TIME RIC AS A DETECTION PLATFORM

xApps subscribe to E2 nodes through service models, of which **. Among these, E2SM-KPM supplies delivers** periodic key performance measurements, and E2SM-RC supplies control primitives [1]. Prototyping platforms including OpenRAN Gym **exposes control actions [1]. Platforms such as OpenRAN Gym [25], SCOPE [26], CoIO-RAN [9] and FlexRIC [9], and FlexRIC [10], together with open radio stacks [27], [28], have made experimental xApp development practical the development of experimental xApps practical. E2 carries aggregated measurements and no packets, so a flow-level detector needs its own data path (Section VII-A).**

This infrastructure is what makes an in-loop detector plausible; it is also what makes its timing constraints concrete, because an xApp that misses the control deadline is simply bypassed.

B. SECURITY OF OPEN RAN

The security implications of RAN disaggregation have been analysed both by **threat model of the O-RAN Alliance's own threat modelling work [?] and by independent studies [29], [30]. These works [31] and several independent analyses [29], [30] identify the RIC and its third-party xApps as a newly exposed attack surface and note the risks introduced by third-party application hosting. Complementary work has implemented and measured new attack surface, and security mechanisms across the O-RAN interfaces [32], and have been implemented and evaluated [32]. For detection, 5G-Spector demonstrates an O-RAN compliant offers layer-3 attack detection service running as a real xApp [11]. Our work is orthogonal in intent: rather than proposing a new detection mechanism, we as an O-RAN compliant service [11], Det-RAN detects attacks from cross-layer data in real time [12], and Fard et al. fuse radio telemetry and flow records on the source corpus we also use [33]. These studies build or evaluate detectors within one deployment. We ask what evidence is needed before any such mechanism can be called deployable would show that such a detector still works in another.**

C. MACHINE LEARNING FOR NETWORK INTRUSION DETECTION

A large family of IoT-oriented benchmark corpora now exists [34]–[38], but cellular-specific corpora are rarer, and **Many benchmark corpora target IoT traffic [34]–[38], and few are cellular. 5G-NIDD is notable for being captured over a functioning was captured on an operational 5G test network rather than in emulation [39], [40] [39], [40], and NetsLab-5GORAN-IDD combines flow records with radio telemetry collected on an O-RAN testbed [41]. For 5G-NIDD, the dataset authors report binary accuracies of 99.85% to 99.95% for four of five classifiers on a random 70/30 split [13], and later work reports a weighted F_1 score of up to 99.95% [14]. Standardized NetFlow feature sets have made cross-corpus experiments feasible [42], [43]. The corpora themselves can be wrong. Labeling**

and flow-construction faults have been documented in CIC-IDS2017 [44], and design flaws that allow a classifier to score well without learning the attack have been found repeatedly across NIDS benchmarks [45].

Feature harmonisation across such corpora has been addressed by standardised NetFlow-based feature sets [42], [43], which is what makes a controlled cross-dataset transfer experiment possible at all.

D. EVALUATION PRACTICE AND ITS CRITICS DATASET SHIFT

A persistent line of critique holds that intrusion **Intrusion** detection results are systematically over-optimistic: Sommer and Paxson identified the mismatch between **often overly optimistic because of** closed-world evaluation and open-world deployment [18]. Axelsson showed that the attack base rate dominates achievable precision [17]. Arp *et al.* catalogued recurring methodological pitfalls including [18], neglect of the base rate [17], sampling bias and inappropriate performance measures [20], and Apruzzese *et al.* examined **data leakage [20], [46], and temporal or spatial bias in the data split [21]. Apruzzese et al. examine the gap between reported and operational effectiveness [47]. Layeghy and Portmann demonstrated empirically that such models generalise poorly across independently collected datasets [15], [16]. Our contribution is across security domains [47], and Kus et al. observe a similar pattern in industrial intrusion detection when detectors are re-evaluated on data on which they were not tuned [48]. The cross-dataset generalization of such detectors has repeatedly been found to be poor [15], [16], [49]. The work most closely related to ours is that of Abraheem and Edhirig, who transfer XGBoost between the same two corpora using 15 harmonized features and report zero-shot balanced accuracies per attack family of 0.50 to 0.56 from NetsLab-5GORAN-IDD to instantiate these critiques within the O-RAN control loop, where an additional and unusually hard constraint — a bounded control deadline — applies alongside them. 5G-NIDD and of 0.52 to 0.83 in the reverse direction [22]. Our transfer results agree with theirs in direction and magnitude. Beyond that study, we add the protocol ladder with input-blind baselines, references measured within the target corpus, the evaluation of a published pipeline under the same ladder, an audit of the target labels, and the operational and latency criteria. From the dataset-shift literature we use label-shift estimation from unlabeled data [50], [51], calibration and its behavior under shift [52]–[54], shift detection [55], and covariance alignment for domain adaptation [56]. For the comparison of classifiers under repeated resampling, we follow Nadeau and Bengio [57].**

III. DEPLOYMENT CRITERIA AND THREAT MODEL

We state the system and threat model, define the three quantities we measure, and combine them into a deployability test.

A. SYSTEM AND THREAT MODEL

We consider an edge data centre hosting a

A near-real-time RIC that serves a set of E2 nodes, as shown in (Fig. 1, with a population of IoT devices attached through those nodes) to which user devices, including IoT devices, are attached. The adversary controls a subset of these devices and uses them to conduct volumetric flooding, reconnaissance scanning, botnet command-and-control communication, or synchronised signalling-plane abuse in the form of rapid connection state cycling perform flooding, scanning, brute-force login, or web attacks, which are the attack categories present in our corpora. The adversary does not control the RIC, the E2 interface, or the xApp itself; platform-level compromise of the RIC is treated by the O-RAN threat model [?] and is out of scope. The defender operates a detection xApp that consumes 's xApp consumes either radio telemetry windows delivered through E2SM-KPM indications and packet-level features, and may issue mitigation actions or flow records produced by a flow exporter co-located with the user plane, and it may act through E2SM-RC. Because E2 does not carry packets, this exporter must sit outside the RIC. Section VII-A discusses its cost and placement.

B. CRITERION 1: CROSS-DEPLOYMENT

GENERALISATION GENERALIZATION ACROSS DEPLOYMENTS

Let $\mathcal{D}_A$ and $\mathcal{D}_B$ be $\mathcal{D}_A$ and $\mathcal{D}_B$ denote two independently collected corpora over represented in a shared feature space $\mathcal{X}$ and label space $\mathcal{Y} = \{\text{benign}, \text{attack}\}$. A model f_θ trained on $\mathcal{D}_A$ is characterised not by characterized by the difference between its held-out score on $\mathcal{D}_A$ alone but by the generalisation gap which upper-bounds nothing on its own but is directly interpretable as the accuracy an operator loses by reusing a published model. We report (1) in both directions and quantify the underlying covariate shift as the mean first-order Wasserstein distance between the marginal feature distributions, where $\mathcal{F}$ is the set of harmonised features and $P^{(j)}$ denotes the marginal and its score on $\mathcal{D}_B$. Because the two corpora differ in attack prevalence (Table 2), a difference in macro- F_1 mixes a change in the classifier's behavior with a change in class balance. Even a classifier that ignores its input obtains a different macro- F_1 when the prevalence changes. We therefore report the difference in balanced accuracy, which is invariant to changes in prevalence,

$$\Delta_{BA} = \text{BA}(f_\theta; \mathcal{D}_A^{\text{test}}) - \text{BA}(f_\theta; \mathcal{D}_B), \quad (1)$$

together with the macro- F_1 gap Δ_{F_1} and its difference from the corresponding gap of the majority-class baseline. Covariate shift is quantified feature by feature as

$$\bar{W}_1 = \frac{1}{|\mathcal{F}|} \sum_{j \in \mathcal{F}} W_1(Q_j(P_A^{(j)}), Q_j(P_B^{(j)})), \quad (2)$$

where Q_j denotes the quantile transform of feature j after per-corpus quantile normalisation, fitted on the source training data only. If each corpus were normalized by its own quantiles, both marginals would be mapped to the uniform distribution, and every term would be zero.

C. CRITERION 2: ALERT BURDEN UNDER A REALISTIC BASE RATE

Let λ_b be the arrival rate of benign flows observed by the xApp and $\text{FPR}(\tau)$ the false positive rate at decision threshold τ . The alert rate presented to the operator is which we report denote the benign arrival rate in the detector's own unit: flow records per hour for a flow-level detector and telemetry windows per hour for a window-level detector. A benign UE generates 225 windows of 16 s per hour. Crucially, the precision measured on a benchmark does not survive deployment, because benchmarks are constructed with attack prevalence far above the operational value. Writing The number of false alerts per hour is

$$A(\tau) = \lambda_b \text{FPR}(\tau), \quad (3)$$

and, for an operational attack prevalence π for the true prevalence of attack flows, the fraction of alerts that correspond to attacks is

$$\text{PPV}(\tau) = \frac{\pi R(\tau)}{\pi R(\tau) + (1 - \pi) \text{FPR}(\tau)}. \quad (4)$$

Neither π nor λ_b can be measured from the corpora, and both are declared and varied over a range. Because PPV depends only on π and $R(\tau)$ for recall, the operational positive predictive value is which for small π is dominated by the second term of the denominator.

Equation (4) is the quantitative form of the base-rate argument [17], and it is the reason a 0.3% false positive rate can still yield a majority-false alert queue.

A scales linearly with λ_b , we also report the number of false alerts per 10^6 benign flows, which is independent of λ_b .

D. CRITERION 3: TAIL LATENCY AGAINST THE CONTROL BUDGET

The end-to-end decision latency of an xApp decomposes as where t_{ind} is E2 indication decoding, t_{feat} can be decomposed as

$$L = t_{\text{ind}} + t_{\text{feat}} + t_{\text{inf}} + t_{\text{act}} + t_q, \quad (5)$$

where the terms are the delivery of the indication to the xApp, feature construction, t_{inf} model inference, t_{act} control message encoding, and t_q queueing delay induced by offered load. inference, the control action up to its acknowledgment, and queueing inside the xApp. We time feature construction and inference alone on one host, and we time all five terms in a loop through a near-real-time RIC with an emulated E2 node (Section V). Conformance is a tail property, not a mean property: for property of the tail of the latency distribution: for a budget B and violation tolerance a tolerated violation rate ε we require with $q_{1-\varepsilon}$ the $(1 - \varepsilon)$ quantile. We adopt $B = 10$ ms, the lower bound of the, we require $q_{1-\varepsilon}(L) \leq B$. Since the O-RAN E2 specification allows near-real-time control loop [?], and $\varepsilon = 0.01$, so (??) reduces to a constraint on the 99th percentile latency requirements anywhere between 10 ms and 1 s [19], we report the smallest budget that is met rather than a verdict for a single chosen value.

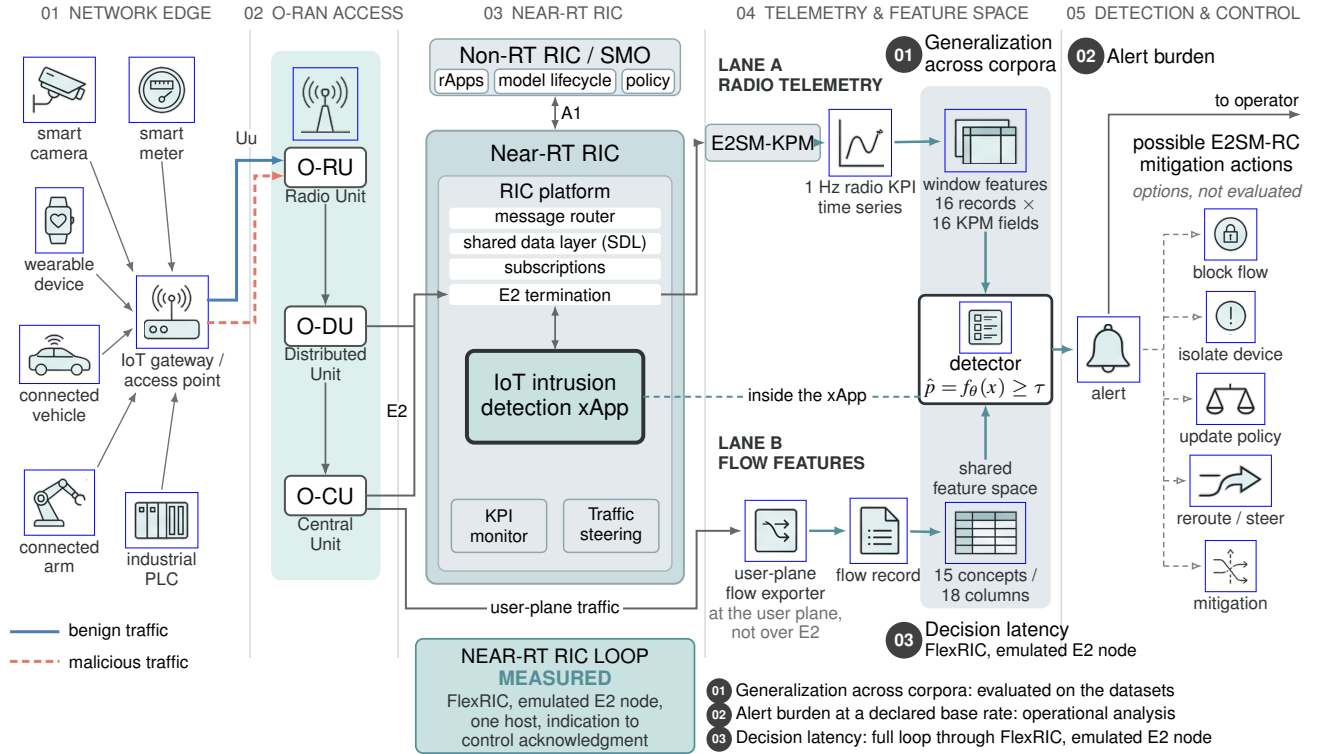


FIGURE 1. Reference O-RAN edge deployment and the detection loop. All device traffic reaches the radio unit over Uu. The detection xApp receives radio telemetry through the E2 termination (lane A) and flow records from an exporter at the user plane, mapped onto the shared feature space (lane B). Column 04 expands the xApp's own feature construction and inference, with one detector per layer. Numbered markers show what this paper measures. Generalization across corpora (01) and alert burden (02) are evaluated on the corpora. Decision latency (03) is timed on one host, from the indication to the acknowledgment of the control message, through a FlexRIC near-real-time RIC with an emulated E2 node. The mitigation actions on the right are options E2SM-RC offers and are not evaluated.

E. A COMBINED DEPLOYABILITY PREDICATE TEST

Taken together, the three criteria define a predicate that a candidate detector either satisfies or does not: The **three criteria are combined into a test that a detector either passes or fails:**

$$\begin{aligned} \mathcal{P}(f_\theta) = & \mathbf{1}[\overline{\Delta}_{BA} \leq \delta] \cdot \mathbf{1}[\overline{q}_{0.99}(L) \leq B] \\ & \cdot \mathbf{1}[\exists \tau : R(\tau) \geq r_{\min}, A(\tau) \leq A_{\max}, \\ & \text{PPV}(\tau) \geq \rho], \end{aligned} \quad (6)$$

where $\overline{\Delta}_{BA}$ and $\overline{q}_{0.99}$ denote upper 95% confidence bounds, so that a pass cannot depend on a favorable point estimate, and all alert conditions are evaluated on the target corpus. The minimum recall $r_{\min}$ prevents a detector that never raises an alert from satisfying the alert term. The operating threshold is then chosen as the most sensitive setting that remains admissible, rather than being left at the conventional $\tau = 0.5$. We stress that (6) is deliberately conjunctive. A detector that satisfies two of the three conditions is not two-thirds deployable; it is undeployable for a specific and nameable reason **admissible threshold**,

$$\begin{aligned} \tau^* = \arg \max_{\tau} R(\tau) \quad \text{s.t.} \quad & R(\tau) \geq r_{\min}, A(\tau) \leq A_{\max}, \\ & \text{PPV}(\tau) \geq \rho. \end{aligned} \quad (7)$$

We propose this test as a reporting instrument rather than as an O-RAN requirement. Its bounds represent

operator requirements, and Section VII-B reports the values used in this study and the sensitivity of the verdict to them.

IV. METHODOLOGY

A. CORPORA

We use two corpora, summarised in Table ??, is an $\mathcal{D}_A$ is **NetsLab-5GORAN-IDD** [41], which was captured on a physical OpenAirInterface O-RAN testbed capture containing packet traces synchronised with per-user radio key performance measurements exported over E2SM-KPM, collected on an OpenRAN Gym style softwareised deployment [25], [26] testbed and contains benign traffic and five attack categories: denial of service (DoS), distributed denial of service (DDoS), probing, brute force, and web attacks. It provides two layers that cannot be joined record by record from the published files: flow records exported by Zeek at the central unit and radio telemetry sampled at 1 Hz at the distributed unit. We use both. The radio layer is aggregated into windows of 16 consecutive records of one UE, each summarized by the mean and standard deviation of 16 KPM fields. Its 30 capture sessions are recovered from gaps of more than five minutes in the timestamps. Fard et al. describe the same corpus as 42 experiment runs [33]. A gap threshold of 90 to 105 s

recovers exactly 42 segments, each with a single attack category, and each of our sessions is a union of complete segments. The five-minute rule merges runs recorded back to back (10 of the 30 sessions contain two or three attack subcategories), so such runs always fall on the same side of a split. Every record of a session carries the same category label. A radio label is thus a session label, and every window of an attack session is an attack window. The sessions follow the capture schedule (Section VI-B), with nine benign sessions on days 0 to 6, the attack categories recorded in blocks on days 47 to 53, and one final benign session. $\mathcal{D}_B$ is 5G-NIDD [40], captured over a functioning [39], [40], which was captured on a 5G test network at a separate institution with a different device population, radio configuration and attack tooling [39]. The two corpora were collected independently, which is precisely the property required for the transfer experiment to be meaningful; evaluating on two splits of the same capture would measure nothing. **different institution and contains flow records exported by Argus with two attack categories, DoS and probing. Its combined files contain no addresses, ports, or timestamps, but they keep the Argus record counter and file offset. The offset returns to its initial value at every file boundary, which yields 20 capture files: two passes over the same ten attack sessions, one per base station. The row index of Combined.csv also restarts at the second pass. A second release of the corpus keeps all Argus fields, including addresses, and the raw captures are published as well (Sections VI-C and VI-E). Each attack file contains benign traffic together with one attack type. A third corpus, $\mathcal{D}_C$, serves as a second transfer target from a third site. It consists of the flow records of a 5G core testbed (Open5GS in Docker) published by Nugraha et al. [24]. Only three of its files contain flow records, each with one attack (a SYN flood, an ICMP flood, and a PFCP session-deletion attack), for a total of 39,425 flows, of which 839 are attacks. Its benign flows comprise user traffic and the signalling of the core network itself. $\mathcal{D}_C$ was exported by NFStream, a third flow exporter, which counts link-layer bytes and, for flows inside the GTP-U user-plane tunnel, the 44 bytes of encapsulation. We remove both over-heads per packet so that the shared columns contain the IP bytes of the inner packet, as they do for $\mathcal{D}_A$. Table 2 summarizes the datasets.**

B. SHARED FEATURE HARMONISATION SPACE

Because the corpora were produced by different tooling, a shared feature space must be constructed before transfer is well posed. is exported by Zeek and by Argus , and the two share *no column names at all*, so the space is defined by meaning rather than by name. The mapping is given in **Zeek and Argus share no column names, so we define the flow feature space by the meaning of each field (Table 3 and is generated from a single committed specification, so that the code and the paper cannot drift apart.**

TABLE 2. Datasets. $\mathcal{D}_A$ supplies training and in-distribution testing on both layers. $\mathcal{D}_B$ and $\mathcal{D}_C$ are transfer targets; each also trains its own in-target references, and $\mathcal{D}_B$ trains the reverse direction.

	$\mathcal{D}_A$ network	$\mathcal{D}_A$ radio	$\mathcal{D}_B$	$\mathcal{D}_C$
Unit	flow	16 s UE window	flow	flow
Source	Zeek	KPM, 1 Hz	Argus	NFStream
Samples	1 640 182	2 808	1 215 889	39 425
Features	18	32	18	18
Attack prevalence	94.63%	76.46%	60.71%	2.13%
Attack categories	5	5	2	3
Groups	318 hosts	30 runs	20 files	3 files
Role	train, test	train, test	target	target

The space contains shared concepts , realised as model). **It has 15 concepts in 18 columns:** seven base numeric quantities each exporter reports natively, one categorical (protocol , expanded against a fixed level list so that source and target matrices cannot differ in width) **counts that both exporters report, a one-hot protocol with a fixed list of levels,** and seven quantities derived identically on both sides. Families available on only one side are excluded and enumerated in the specification; in particular publishes no inter-arrival statistics, so no timing family exists in the shared space, and 's radio KPIs have no counterpart and are examined separately.

One mapping deserves stating because the obvious choice is wrong **derived quantities computed the same way for both corpora. One mapping needs care.** Zeek's `src_bytes` counts application payload, and both corpora are dominated by **which is zero for most floods and scans**that carry none, so its median is zero; , **whereas** Argus's `SrcBytes` counts the whole frame and has a median of 84. Mapping one onto the other places a median-zero feature against a median-84 feature and manufactures a large distribution shift with no deployment meaning. The counterpart that actually corresponds is `src_ip_bytes`, at **complete packets. The correct counterpart of `SrcBytes` is Zeek's `src_ip_bytes`, with 40 against and 42 bytes per packet, the residual being link-layer framing that Argus includes and Zeek does not. respectively. A lower bound on duration and the transform $\log(1 + x)$ for unbounded quantities are applied identically to both corpora, and scaling is fitted on the source training data only. No address, port, or timestamp is used as a feature.**

Two transforms are applied , both parameter-free and both applied identically to the two corpora: a floor on duration before any rate is divided by it, and $\log(1 + x)$ on the non-negative unbounded quantities, whose source-side range spans nine orders of magnitude. Scaling is fitted *on source training data only*, inside the model pipeline, so that no statistic of the target corpus can enter source-side preprocessing. No feature derived from absolute timestamps or endpoint addresses is retained, since such features leak capture identity rather than attack behaviour [20]; in fact publishes no addresses or ports at all. **Before any result was computed with it, we also defined an exporter-robust subset of nine columns, consisting of the protocol and the five per-packet means and direction ratios. The two exporters differ in where they terminate a flow, which affects duration, rates, and totals. Per-packet means and direction ratios are less affected, but they are not invariant. Splitting a flow changes them whenever its packets differ, for example when handshake and teardown packets are assigned to different records, and the two exporters count the same packet as 40 and 42 bytes, an offset of about 5% for minimum-**

TABLE 3. Shared flow feature space, generated from `configs/features/shared_space.yaml`. The exporter-robust subset is the protocol one-hot, the three mean packet sizes, and the two direction ratios.

Concept	$\mathcal{D}_A$ (Zeek)	$\mathcal{D}_B$ (Argus)	Role
duration	duration	Dur	base
src_bytes	src_ip_bytes	SrcBytes	base
dst_bytes	dst_ip_bytes	DstBytes	base
src_pkts	src_pkts	SrcPkts	base
dst_pkts	dst_pkts	DstPkts	base
tot_pkts	src_pkts + dst_pkts	TotPkts	base
tot_bytes	src_ip_bytes + dst_ip_bytes	TotBytes	base
proto	proto	Proto	one-hot (4)
mean_pkt_size	tot_bytes / max(tot_pkts, 1)		derived
src_mean_pkt_size	src_bytes / max(src_pkts, 1)		derived
dst_mean_pkt_size	dst_bytes / max(dst_pkts, 1)		derived
bytes_per_s	tot_bytes / max(duration, duration_floor_s)		derived
pkts_per_s	tot_pkts / max(duration, duration_floor_s)		derived
src_byte_ratio	src_bytes / max(tot_bytes, 1)		derived
src_pkt_ratio	src_pkts / max(tot_pkts, 1)		derived

size packets.

C. DETECTOR FAMILY AND OBJECTIVE DETECTORS

We evaluate eight architectures, and they are *subjects rather than competitors*: no claim here is that one beats another, only that a particular thing happens to all of them. Six are non-trivial — logistic regression, a depth-limited decision tree, a random forest [58], two independent gradient-boosting implementations [?], [59], **Six supervised architectures** and a multilayer perceptron. The remaining two are trivial floors: **two trivial baselines are evaluated under every protocol. The supervised models are LR, a DT of depth 12, an RF [58], XGB [59], the HGB implementation of scikit-learn [60], and an MLP, and they are compared with a majority-class predictor and a stratified sampler of the training prior.**

The floors are not decoration. On a corpus that is % attack, a majority-class classifier achieves that accuracy by construction, and without the floor on the page no other number on it can be read.

They also serve as a control: a classifier with no capacity to memorise cannot benefit from a leaky evaluation protocol, so if the floors gain nothing where the others gain, the mechanism is memorisation. **Because the baselines ignore their input, any change in their score between two protocols reflects a change in class composition alone. All supervised models use balanced class weights, applied as sample weights for the MLP and through `scale_pos_weight` for XGB. The hyperparameters are fixed and were not tuned (Table 4). Tuning on held-out source data would favor source-specific structure, the very effect the transfer experiments measure, and tuning on the target would leak target information into training.**

Every architecture receives the same preprocessing, the same class-imbalance policy and the same tuning budget. Under-tuning the more flexible models would inflate every gap we report and is the first thing a reviewer should attack. **Three further model families answer specific questions. Two sequence models, a GRU [61] and a one-dimensional convolutional network, operate on the raw 16-step KPM sequences of the same radio windows. Two unsupervised detectors, an isolation forest [62] and an autoencoder, are trained on benign source flows**

TABLE 4. Hyperparameters, identical across protocols and seeds.

Model	Settings
Majority, Strat.	training majority; samples of the training prior
LR	<code>max_iter</code> 2000, balanced weights, standardized
DT	depth 12, balanced weights
RF	200 trees, min. leaf 2, balanced subsample weights
XGB	300 trees, depth 6, $\eta = 0.1$, subsample 0.8, <code>hist</code>
HGB	300 iterations, depth 6, $\eta = 0.1$, balanced weights
MLP	128–64, Adam, early stopping (patience 10), balanced sample weights, standardized
GRU	hidden 32; BCE with positive weight n_-/n_+
1D-CNN	two convolutions (32, kernel 3), average pooling
IF	200 trees; benign-only training
AE	12–6–12 MLP, reconstruction error; benign-only
CORAL	per-domain z-score, identity-regularized covariances

only, and their alarm threshold is fixed at the 99th percentile of the scores of benign source flows. A CORAL variant [56] re-colors the source flows to the covariance of the unlabeled target before training, the only case in which a detector sees target data.

D. EVALUATION PROTOCOLS

Splits are group-disjoint wherever a group key is available, and an automated check rejects any split in which a group appears on both sides. On the radio layer, the group key is the capture session, and three protocols are compared: a random split of windows, a run-disjoint split, and a category-stratified run-disjoint split that keeps sessions of every attack category on both sides, so that no test category is absent from training. Two further designs address the effect of time (Section VI-B). In the first, the latest or the earliest session of every category is held out and compared with draws that hold out a random session of every category. In the second, each benign session is held out individually. On the flow layer, the group key is the source address (318 groups). For each of 20 split seeds, a subsample of 300, so uniformity of treatment is the defence rather than a per-model search. Class imbalance is handled with a weighted binary cross-entropy objective, with $\hat{p}_i = f_\theta(x_i)$, weights $w_\pm$ set inversely to class frequency, and γ the regularisation coefficient. The tree ensembles use the equivalent instance-weighting mechanism. 000 flows is drawn from $\mathcal{D}_A$ and divided by source address. Every model is then evaluated on the whole of $\mathcal{D}_B$, whereas the reverse direction trains on a random split of $\mathcal{D}_B$ and evaluates on $\mathcal{D}_A$. Within $\mathcal{D}_B$ itself, the in-target references either hold out four of the 20 capture files or train on the pass of one base station and test on the pass of the other (Section VI-C). Apart from these references, CORAL, the domain classifier, and the prior estimates of Section VI-H, which are marked as such, no transform, threshold, or model choice uses $\mathcal{D}_B$. Every access to it is logged.

E. EXPERIMENTAL PIPELINE STATISTICAL ANALYSIS

Fig. 2 shows the pipeline. A single trained model is carried unchanged through three evaluations: an in-distribution test split, a cross-deployment evaluation on , and a runtime evaluation as a packaged xApp. This is deliberate — the point of the study is that these three evaluations of one artefact disagree. **Uncertainty is estimated over 20 split seeds. Because the splits resample a single population of 30 sessions or 318 source addresses, their training and test sets overlap, and a paired t -test over split means underestimates the variance. We therefore report Nadeau–Bengio corrected intervals [57], which scale the variance by $1/(J+n_{\text{test}}/n_{\text{train}})$ for J splits. We apply Holm correction within each family of tests. A family is one metric and one comparison (a protocol pair or a transfer direction) across the six supervised architectures, and the two sequence models form a family of their own. Operating points come from confusion counts pooled over folds, and Section VI-G explains why per-fold precision should not be averaged. Pooled counts include a sample once for every seed in which it is tested, and samples within a cluster are correlated, so a binomial interval would be far too narrow. We use a cluster bootstrap with 2,000 replicates instead. It resamples with replacement both the split seeds and the clusters: radio sessions, $\mathcal{D}_A$ source addresses, or $\mathcal{D}_B$ capture files.**

F. EXPERIMENTAL PIPELINE AND INFERENCE LOOP

G. INFERENCE LOOP

Fig. 2 summarizes the experimental pipeline, and Algorithm 1 gives the xApp control loop decision loop of a detection xApp. A decision that exceeds the budget is still acted upon and recorded as a violation, since a late mitigation is usually better than none. An operator who prefers to drop late actions can move the transmission step inside the deadline check. Our FlexRIC xApp (Section V) runs this loop with two changes made for timing. It scores held-out test windows instead of windows built from the synthetic KPM values, and it sends a control message after every report, whatever the score, so that t_{act} is measured on every report.

Two details matter for the latency results. First, feature construction is incremental: the window statistics are updated per indication rather than recomputed, which is what keeps t_{feat} in (5) bounded. Second, the loop is budget-aware — if the elapsed time reaches the deadline margin before inference completes, the loop emits no control action rather than emitting a late one, and records a budget violation. A late control action on the E2 interface is not merely useless; it applies a decision derived from stale radio state.

V. EXPERIMENTAL SETUP

Every measurement in this paper was produced on one commodity workstation : an Intel Core i-series host running Windows . **All measurements were performed on a single workstation with a 13th Gen Intel Core i9-13900H processor (14 cores, 20 threads), 47.6 GB of memory, and Windows 11 Pro build 26120, using Python 3.11.9, scikit-learn 1.4 and XGBoost 2.0, with inference on CPU. A full pip freeze of the interpreter that produced every number is committed alongside 1.8.0, XGBoost 3.2.0, NumPy 2.3.5, and ONNX Runtime 1.30.0. The sequence models were trained**

Algorithm 1 Budget-aware decision loop of a detection xApp

Require: indication stream $\mathcal{S}$, model f_θ , threshold τ , budget B , margin δ_m
Ensure: E2 control actions; violation count v ; latency record

```

1:  $\mathcal{W} \leftarrow$  empty per-UE window map;  $v \leftarrow 0$ 
2: for all indication  $m$  arriving on  $\mathcal{S}$  do
3:    $t_0 \leftarrow \text{Now}()$ ;  $u \leftarrow \text{UEOf}(m)$ 
4:    $\mathcal{W}[u] \leftarrow \text{UpdateWindow}(\mathcal{W}[u], m)$ 
5:    $x \leftarrow \text{Features}(\mathcal{W}[u])$ 
6:    $\hat{p} \leftarrow f_\theta(x)$ 
7:   if  $\hat{p} \geq \tau$  then
8:      $\text{SendE2}(\text{EncodeControl}(u, \hat{p}))$   $\triangleright$  even if late
9:   end if
10:   $\ell \leftarrow \text{Now}() - t_0$ 
11:  if  $\ell > B - \delta_m$  then
12:     $v \leftarrow v + 1$   $\triangleright$  deadline violation, recorded
13:  end if
14:   $\text{RecordLatency}(\ell)$   $\triangleright$  every indication
15: end for
```

with PyTorch 2.14 on the CPU in a separate environment. A complete list of packages is provided with the results.

No RIC was involved. We did not deploy the detector as an xApp, we did not run an E2 agent or a RAN stack, and we did not containerise anything. The latency figures in Section VI are *emulated*: they time the decision path in a single Python process on a host with no CPU isolation. They support per-stage decomposition and relative ordering between implementations, and they support no conformance claim whatever. A measurement of AI inference inside a real Near-RT RIC exists in the literature [23] and reports figures three orders of magnitude below ours for the same model families. **For the latency loop, we built FlexRIC [10] v2.0.0 in WSL2 (Ubuntu, Linux kernel 6.18) on the same workstation and connected its near-real-time RIC over SCTP to the emulated gNB agent that ships with FlexRIC. Our xApp, written in C, subscribes to E2SM-KPM reports with 16 measurement items per UE, which matches the size of one radio record. It scores every UE in a report with ONNX Runtime 1.30.0 through its C API and answers each report with one E2SM-RC control message (radio bearer control, QoS flow mapping), which is the comparison we draw rather than a substitute for it.**

is split group-disjointly: whole groups go to exactly one side, and a guard raises if a group is ever found on both. The grouping key is the capture run on the radio layer (recovered runs) and the source address on the network layer. Device-disjoint splitting, which an earlier version of this protocol specified, is **not supportable on this corpus**: ue_id takes nine values and one of them holds % of records, so a device-disjoint split is not a split. **only control the emulated node implements. It then waits for the acknowledgment. The emulated node reports synthetic values for 0 to 12 UEs, so the xApp scores held-out test windows in their place. Before timing, the C implementation reproduced the Python scores on all 589 test windows to within 4×10^{-7} . We take the delivery time from the timestamp that the node writes into the KPM header, since all processes share one clock. No radio, no real E2**

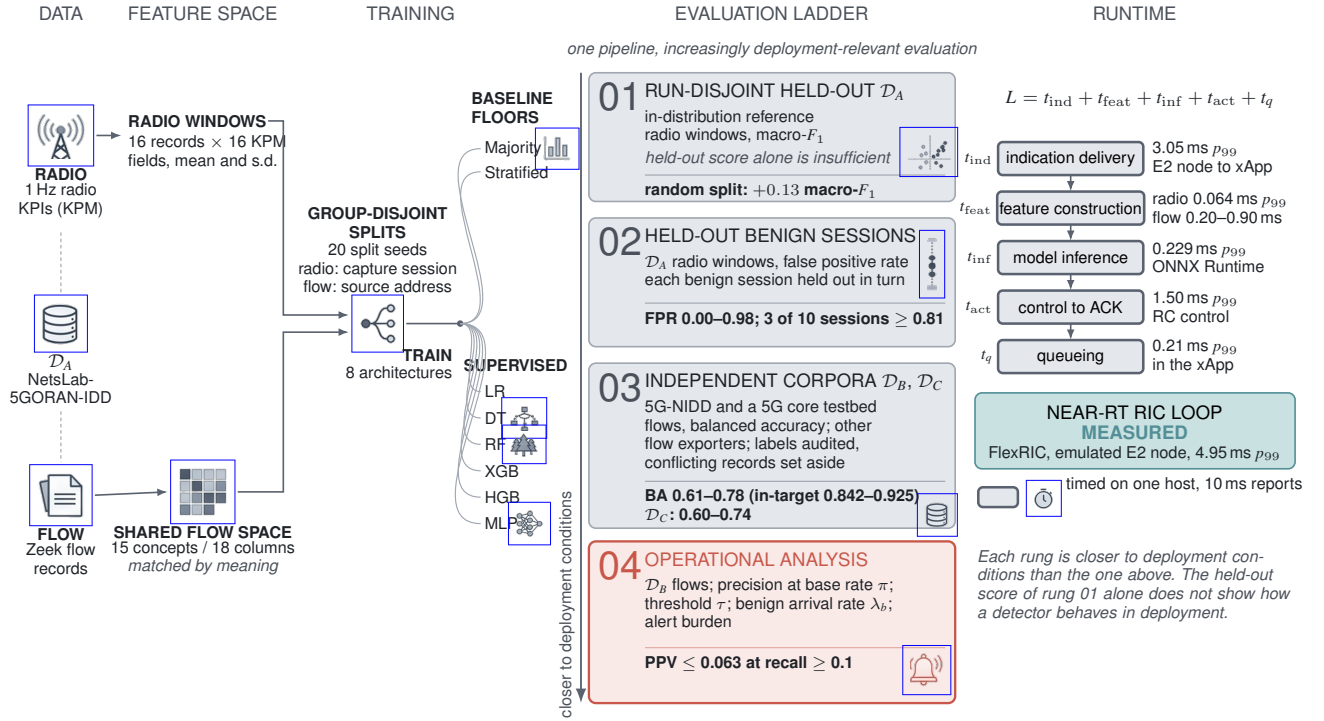


FIGURE 2. Experimental pipeline and evaluation ladder. Radio windows and flows from $\mathcal{D}_A$ are summarized separately, split by group, and used to train eight architectures. Each rung of the ladder moves closer to deployment conditions: a run-disjoint held-out split, held-out benign sessions, two independently collected corpora evaluated on flows with consistent labels, and an operational analysis at a declared base rate. The bold line under each rung gives the measured result. The runtime column gives the 99th percentile of each term of Eq. (5), measured on one host through a FlexRIC near-real-time RIC with an emulated E2 node at a 10 ms reporting period. Flow feature construction was timed without a RIC.

node, and no container platform took part, and the remaining latency figures were measured without a RIC.

Uncertainty is reported over split seeds, as paired t -intervals over split means rather than as percentile bootstraps. The choice is not stylistic: at small n a percentile bootstrap resampling a handful of split means can only span the observed values, so it understates uncertainty, and in an earlier version of this work it reported four significant effects that the correct paired test does not support. The two-level seed design is deliberate, because the dominant variance is which groups land in which fold, not which random state the model used.

is never used for training, for model selection, for fitting any transform or for choosing any threshold, and it is consumed once, at final evaluation. Every read of it is timestamped in an append-only access log so that this is auditable after the fact rather than asserted. Hyperparameters are given in Table 4; none was tuned per architecture.

Latency is timed in-process. The comparison of protocols on the radio layer uses 20 split seeds with two model seeds each, and the remaining radio-layer and flow-layer experiments use 20 split seeds with one model seed. Calibration uses 10 split seeds of a three-way source split into training, calibration, and test groups. The declared operational parameters are an attack prevalence of $\pi = 0.002$ and a benign arrival rate of $\lambda_b = 240,000$ flows per hour, and both are varied over a range. For the radio layer, false alerts are reported per benign UE-hour. Neither parameter comes from an operator. The chosen rate of about 67 flows per second is below the several hundred

flows per second that Jin et al. expect at a heavily loaded base station of a large LTE network [63], so our alert volumes probably understate those of an edge site serving several cells. Latency is measured per call with a monotonic clock around each stage of the decision path. Each architecture is warmed up for 200 iterations and then timed over, with garbage collection enabled, after 1, 500, and we report p_{50} , p_{95} , p_{99} and $p_{99.9}$ rather than a mean. A no-op detector is timed first, so that the platform floor is known before any model is measured: its p_{99} is ms, two orders of magnitude below the tightest budget we consider, which is what makes the model figures interpretable at all. We report no throughput or offered-load sweep, because we ran none. 000 warm-up calls, over 20,000 calls, and with one thread per library. Distribution-free confidence intervals are reported for every quantile.

For the alert-burden measurement we use a benign-only capture segment from the deployment at an aggregate rate of $\lambda_b = 240,000$ flows per hour, corresponding to a modest regional edge site. Operational attack prevalence is taken as $\pi = 0.002$; this is not a measured value but a stated modelling assumption,

VI. RESULTS

We follow the evaluation ladder: protocol sensitivity within one corpus, time order and benign sessions, the target corpus and a published result on it, transfer across corpora and the composition of its gap, and Section IX discusses its sensitivity. a third corpus. Alert burden, calibration, and latency follow.

VII. RESULTS AND DISCUSSION

A. EVALUATION-PROTOCOL SENSITIVITY

B. IN-DISTRIBUTION PERFORMANCE

Table 5 and Fig. 3 compare three protocols on the radio layer. Relative to a run-disjoint split, a random split raises macro- F_1 by 0.09 to 0.17 across the six architectures, or by 0.13 on average, and the gain is positive in at least 16 of the 20 splits for every architecture. The trivial baselines show no gain. The corrected interval for the average gain is $[-0.01, 0.27]$ ($p = 0.06$). Thirty capture sessions are too few to estimate the size of the effect precisely, and no single architecture's gain survives Holm correction. An uncorrected paired t -test would give $p = 6 \times 10^{-5}$ on the same data.

Before any transfer question can be posed, the in-distribution number has to be pinned to a protocol, because the protocol moves it more than the architecture does. Table ?? reports Missing test categories do not explain the gain, since every test category also appeared in training in all 20 run-disjoint splits. The category-stratified protocol guarantees this by construction and yields a gain of 0.14 to 0.19. It also moves sessions of rare categories into the test set, which makes its test sets harder, so we do not read its larger gain as further evidence. A model-free probe points to the mechanism. Under a random split, 68% of the test windows have their nearest training window in the same capture session (65% to 71% across splits), and a one-nearest-neighbor (1-NN) lookup achieves a macro- F_1 on under two protocols that differ only in how the split is drawn: F_1 of 0.96, compared with 0.81 under a run-disjoint split (Table 5). Every radio label is also a session label, so recognizing the session is enough to predict the label, and a random split lets any model exploit this, a lookup table included. Under the run-disjoint split, the lookup beats LR and DT and comes within 0.04 of the best supervised architecture. On this layer, the learned models seem to add little to the retrieval of similar windows. The two sequence models in Table 6, which operate on the raw 16-step KPM sequences instead of window summaries, exhibit the same pattern more clearly. They achieve 0.923 to 0.944 under a random split over flow records, which is what most of this literature reports, and 0.755 to 0.759 under a run-disjoint split in which whole capture runs go to exactly one side, a gain of 0.16 to 0.19 whose corrected intervals exclude zero for both models (Holm-adjusted $p \leq 0.02$ within the pair).

The difference is large and it is not a constant offset. It spans to of macro- F_1 ,

B. TIME ORDER AND BENIGN SESSIONS

Although the radio layer spans 52.96 days, its capture order follows the scenario schedule (Table 7). Nine benign sessions were recorded on days 0 to 6, the five attack categories were recorded in blocks on days 47 to 53, and a final benign session was

TABLE 5. Protocol sensitivity on the $\mathcal{D}_A$ radio layer: macro- F_1 under random, run-disjoint (RD) and category-stratified run-disjoint (CS) splits, and paired gains of the random split with Nadeau-Bengio corrected 95% intervals over 20 split seeds. No supervised architecture's gain is significant after Holm correction. 1-NN: a model-free nearest-neighbor lookup on the same splits.

Model	Random	Run-disj.	Cat.-strat.	Δ_{R-RD} [95% CI]	Δ_{R-CS} [95% CI]
LR	0.836	0.744	0.697	+0.093 [-0.05, +0.24]	+0.139 [-0.06, +0.33]
DT	0.950	0.782	0.758	+0.168 [+0.02, +0.32]	+0.192 [+0.02, +0.36]
RF	0.970	0.853	0.816	+0.117 [-0.04, +0.27]	+0.154 [-0.02, +0.32]
XGB	0.977	0.837	0.808	+0.140 [-0.00, +0.28]	+0.169 [-0.02, +0.35]
HGB	0.982	0.844	0.831	+0.138 [-0.01, +0.28]	+0.152 [-0.03, +0.33]
MLP	0.954	0.828	0.809	+0.126 [-0.01, +0.26]	+0.146 [-0.02, +0.31]
Strat.	0.498	0.507	0.500	-0.009 [-0.01, -0.00]	-0.002 [-0.01, +0.01]
Maj.	0.431	0.434	0.447	-0.002 [-0.02, +0.01]	-0.016 [-0.04, +0.01]
1-NN	0.958	0.814	0.794	+0.144 [+0.00, +0.28]	+0.164 [+0.01, +0.31]

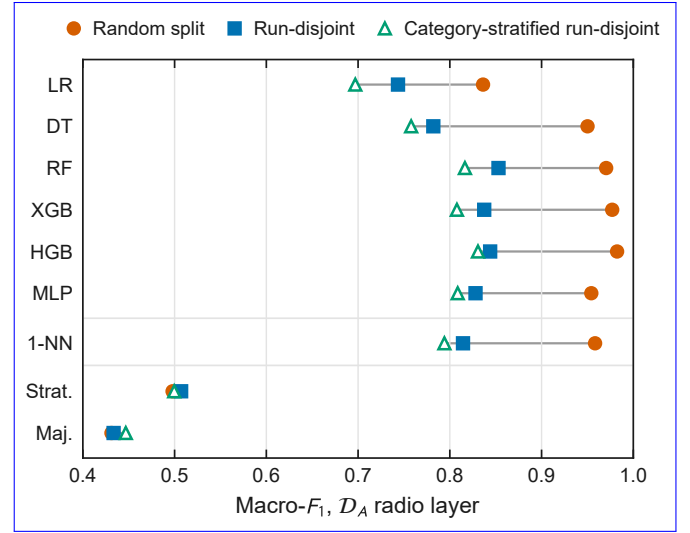


FIGURE 3. Macro- F_1 on the $\mathcal{D}_A$ radio layer under three protocols, mean over 20 split seeds; the gray line spans the three protocols of each model. The random split increases the score of every supervised model and of the model-free 1-NN lookup, whereas the scores of the input-blind baselines remain unchanged. Retaining every attack category in training (category-stratified split) does not eliminate the decrease. Corrected intervals are given in Table 5.

recorded on day 53. Each session is either benign or belongs to one attack category throughout, so a split by session order holds out entire attack categories. For training fractions between 0.4 and it tracks model flexibility:

a single depth-12 tree gains most, the boosted ensembles next, the linear model least. The two trivial baselines gain nothing at all, and that is the control which identifies the mechanism. A classifier with no capacity to memorise cannot profit from a leaky split, so the gain is memorisation rather than a change in task difficulty.

The consequence for a reader of this literature is direct: a published random-split score is inflated *most* for exactly the flexible architectures most likely to be published, so the inflation cannot be mentally subtracted as a fixed correction. 0.7, the latest sessions contain 1 to 4 attack categories absent from training, plus a single benign session. Such a split measures the detection of unseen categories, not temporal drift, and we do not report it as a drift estimate.

To keep category coverage fixed, we held out the

TABLE 6. Sequence models on the same radio windows and splits as Table 5.

Model	Random	Run-disj.	Cat.-strat.	Δ_{R-RD} [95% CI]
1D-CNN	0.944	0.755	0.763	+0.189 [+0.05, +0.32]
GRU	0.923	0.759	0.769	+0.165 [+0.04, +0.29]

TABLE 7. False positive rate on each benign radio session when that session alone is held out and the models train on all other 29 sessions, mean and range over the six architectures. Day: capture start relative to the first session. Alerts per benign UE-hour: $FPR \times 225$ windows.

Session	Day	UEs	Windows	FPR	Range	Alerts/UE-h
0	0.0	1	57	0.00	[0.00, 0.00]	0
1	0.0	1	54	0.02	[0.00, 0.06]	6
2	2.8	1	58	0.00	[0.00, 0.00]	0
3	2.9	1	55	0.07	[0.02, 0.20]	15
4	3.0	1	24	0.81	[0.54, 0.96]	181
5	3.9	1	56	0.14	[0.00, 0.77]	31
6	4.0	2	60	0.00	[0.00, 0.00]	0
7	4.0	1	95	0.01	[0.00, 0.03]	2
8	6.0	1	56	0.98	[0.93, 1.00]	220
29	52.9	3	146	0.87	[0.84, 0.93]	196

latest session of every category, including the benign category, and compared the result with 50 control draws, each of which holds out a randomly chosen session of every category (Table 8). When the latest sessions are held out, balanced accuracy ranges from 0.52 to 0.57, compared with control means of 0.75 to 0.83, and yet 12% to 32% of the control draws score equally low. When the earliest sessions are held out, balanced accuracy ranges from 0.95 to 0.99, a level that 4% to 34% of the control draws match or exceed. The effect of time order thus remains within the ordinary variation between sessions.

Most of this variation comes from the benign session that lands in the test set. When the benign sessions are held out one at a time (Table 7 and Fig. 4), 6 of the ten sessions yield a false positive rate of at most 0.07, whereas sessions 4, 8 and 29 (days 3, 6 and 53) yield rates between 0.81 and 0.98. The final benign session, the only one recorded during the attack period, yields a false positive rate of 0.87 (196 false alerts per benign UE-hour), but two sessions from the first week yield similar rates. Ten benign sessions cannot separate a change over time from this heterogeneity. The false positive rate of a detector at a new site therefore cannot be inferred from its rate on the benign sessions it was tested on.

C. CROSS-DEPLOYMENT GENERALISATION

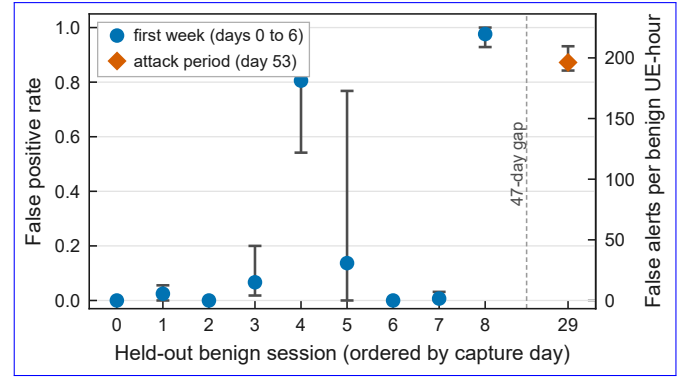
C. THE TARGET CORPUS: LABELS AND A PUBLISHED RESULT

Label conflicts. Before evaluating transfer to $\mathcal{D}_B$. Every non-trivial architecture degrades, and all six differences survive Holm correction across the family. That is the less informative, we examined whether its labels are a function of its flow records, and found

TABLE 8. Category-stratified time split on the $\mathcal{D}_A$ radio layer: balanced accuracy when the latest or the earliest session of every category is held out, against the mean of 50 draws that hold out a random session of every category. "% ctrl. $\leq$ ": share of control draws scoring at or below the time split. Every test category is in training.

Model	Control		Latest held out		Earliest held out	
	BA	% ctrl. $\leq$	BA	% ctrl. $\leq$	BA	% ctrl. $\leq$
LR	0.749	0.573	0.521	12	0.950	96
DT	0.774	0.521	0.521	12	0.962	84
RF	0.811	0.562	0.521	32	0.970	66
XGB	0.812	0.567	0.521	26	0.978	72
HGB	0.817	0.550	0.521	32	0.995	94
MLP	0.827	0.574	0.521	18	0.983	94

Table ?? shows what happens when the identical models are applied

**FIGURE 4.** False positive rate on each benign radio session when that session alone is held out: mean (marker) and range (bar) over the six architectures, sessions in capture order. The right axis gives the same rate as false alerts per benign UE-hour (225 windows). Sessions recorded in the same week range from no false positives to almost all windows flagged, which indicates that the false positive rate depends more on the particular benign session than on the time of capture.

that this is not the case for half of the result.

The informative half is the floor. A macro- F_1 of 0.53 carries no meaning until one knows what guessing scores on the same corpus, and on corpus. Of the 1,215,889 flows, 52% (632,378) share every field except the row index, Seq, and Offset with a flow that carries the opposite label (Table 9, Fig. 5). All of these flows lie in the two UDP-flood captures, where they account for 75% of file 5 and 98% of file 15, one file from each base station. The match is exact. Each of the 281,529 flows labeled as UDP flood in file 15 has an identical copy labeled as benign in file 5, and the counts are equal for 33,704 of the 33,708 distinct records. These copies constitute 59% of the benign class of the corpus. The combined files carry no addresses, but the release of the corpus with all fields preserved does, and it aligns with them row for row. All 281,525 benign copies are UDP flows from 10.155.15.7 to 10.41.150.68. The same host pair, protocol, and time window form the flood labeled as attack in file 15, whereas the attack labeled in file 5 comes from another host, 10.155.15.4, and targets the same address. The capture at the first station thus also recorded the flood of the attacker at the second station, and only its own attacker was labeled. We

therefore regard the benign copies as mislabeled attack traffic, which makes the evaluation without them the one with correct labels. We change no label and report every $\mathcal{D}_B$ a stratified random classifier scores. The best architecture clears that by ; the MLP scores , which is *beneath* the floor, with **result both with and without the copies.**

These conflicts cap the performance of any classifier that reads the record content. Even on the flows it was built from, the best possible function of the 90 content fields reaches a balanced accuracy of — on the wrong side of chance. Trained on one O-RAN deployment and applied to another, it does worse than guessing, and it was the strongest model on held-out source data **0.766**. In the 18 shared columns the bound is 0.753, the level reached by DT, XGB, HGB, and the MLP when they are trained and tested on a random split of $\mathcal{D}_B$ (Table 10). When *Seq* and *Offset* are added, every record becomes unique and the bound rises to 1.000.

That last observation generalises. The architecture ranked first on the source ranks *last of six* on the target, and across the six the rank correlation between in-distribution and transfer performance is not significant (Spearman $\rho = 0.143$, $p = 0.787$). A practitioner selecting an architecture by its published in-distribution score therefore has close to no information about how it will transfer — a stronger statement than the usual observation that scores drop. **References within the target.** To separate the cost of transfer from the limits of $\mathcal{D}_B$ itself, we also trained and tested the six architectures on $\mathcal{D}_B$ alone (Table 10). On a random split, they reach 0.708 to 0.752 in the shared space, the bound given above, and 0.842 to 0.925 when the benign copies are excluded from the test set. On held-out capture files, they reach 0.931 to 0.982 in the shared space and 0.990 to 1.000 with native features, although none of the three draws held out a UDP-flood capture. Across the two base stations, within a single corpus and with a single exporter, balanced accuracy decreases to 0.618 to 0.641 from station 1 to station 2 and to 0.639 to 0.650 in the opposite direction. Both decreases are caused by the copies. A model trained on station 1, where the flood copies are labeled benign, misses the floods of station 2, whereas a model trained on station 2 flags the benign-labeled copies of station 1 and scores 0.973 to 0.988 when these copies are removed from the test set. On flows whose labels follow from the record content, the shared space separates the classes of $\mathcal{D}_B$ almost perfectly, so the target corpus itself does not limit transfer.

A published result under the same ladder. The dataset authors report binary accuracies of 99.85% to 99.95% on a random 70/30 split [13]. We reproduced their pipeline, which consists of the published one-hot encoding, selection of the ten features with the highest ANOVA F -scores, z -score normalization, and their five classifiers, for which we used the scikit-learn default settings because the tuned values are not reported. Our F -scores rank the features

TABLE 9. Label conflicts in $\mathcal{D}_B$ (5G-NIDD). Content: all fields of the published encoding except row index, *Seq*, *Offset*, and the labels. Ceiling: balanced accuracy of the best lookup on distinct records, in sample.

Features	Cols	Records	% in conflicting records			BA ceiling
			All	Benign	Attack	
Content fields	90	374 946	52.0	58.9	47.5	0.766
Shared space	18	246 281	62.4	59.7	64.1	0.753
Content + <i>Seq</i> , <i>Offset</i>	92	1 215 869	0.0	0.0	0.0	1.000

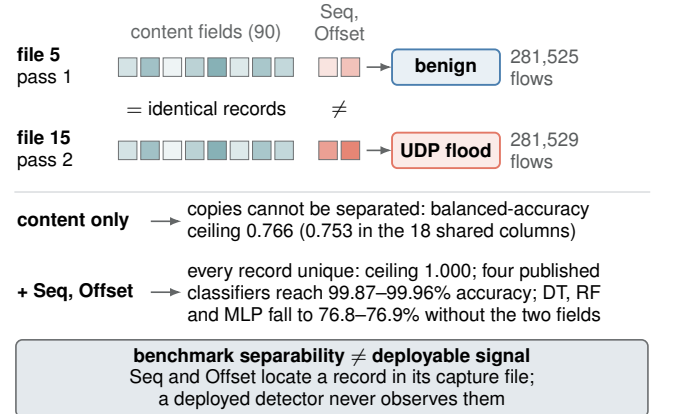


FIGURE 5. Illustration of the label conflict in $\mathcal{D}_B$. Identical flow records occur in two capture files from the two passes and are labeled benign in one file and UDP flood in the other (the tiles are schematic). Because no function of the record content can separate the copies, the attainable balanced accuracy is bounded. The record-position fields *Seq* and *Offset* differ between the copies, make every record unique, and account for the published accuracy of 99.9%, although a deployed detector cannot observe them.

in exactly the same order, and on their split we obtain accuracies of 99.87% to 99.96% for DT, RF, KNN, and the MLP (Naive Bayes 92.71%), in agreement with their results (Table 11). Their first and second features are *Seq* and *Offset*, which encode the position of a record within its capture file. A deployed detector cannot observe these fields, and, according to the bounds given above, they are what separates the conflicting records. Replacing only these two fields with the next two features in their ranking reduces the accuracy of DT, RF, and the MLP to 76.8% to 76.9% (balanced accuracy 0.704 to 0.706, false positive rate 0.588 to 0.591). This decrease reflects the label conflict rather than a loss of attack information, since RF still achieves 0.9999 on the quantitative form of the exporter confound declared in Section V. test flows without the benign copies. The two counters thus allow the published models to distinguish identical records by their position within the file. When capture files are held out, the published features yield average balanced accuracies of 0.859 to 0.923 for DT, RF, and the MLP, with RF ranging from 0.82 to 1.00 across folds. Without the counters, the average is 0.799 to 0.800, RF ranges from 0.50 to 1.00, and accuracy falls to 1.6% on the fold that tests one

TABLE 10. References inside $\mathcal{D}_B$: balanced accuracy of models trained and tested on $\mathcal{D}_B$ only, mean over n draws of a 240,000-flow training sample. *Range over the six architectures on the test flows without the benign copies of Table 9. Capture-file draws hold out four of the 20 files.

Features	Split	LR	DT	RF	XGB	HGB	MLP	BA*	n
Shared (18)	Random	0.708	0.752	0.723	0.752	0.752	0.750	0.84–0.92	3
Shared (18)	Capture file	0.955	0.982	0.940	0.943	0.956	0.931	0.93–0.98	3
Shared (18)	Station 1 to 2	0.618	0.638	0.640	0.641	0.640	0.635	0.62–0.64	1
Shared (18)	Station 2 to 1	0.639	0.648	0.648	0.648	0.648	0.650	0.97–0.99	1
Native	Random	0.710	0.764	0.735	0.764	0.764	0.764	0.80–0.87	2
Native	Capture file	0.990	1.000	0.999	1.000	1.000	0.994	0.99–1.00	2
Native	Station 1 to 2	0.698	0.661	0.662	0.662	0.662	0.662	0.66–0.70	1
Native	Station 2 to 1	0.648	0.648	0.649	0.650	0.650	0.654	0.99–1.00	1

How separable are the two corpora in the shared space? We do not answer this with a covariate-shift statistic of our own, because a stronger answer already exists: Abraheem and Edhirig [22] train a classifier to predict *which corpus* a flow came from and reach a balanced accuracy of 0.993 on harmonised features of the same two corpora, a figure that survives correlation alignment and feature filtering. The corpora are trivially separable. That bounds what any transfer measurement between them can mean,

flood capture after training on the other. Across base stations, the published features yield 0.762 to 0.895, and the features without counters yield 0.656 to 0.692. A published accuracy of 99.9% on this corpus says little about a new capture file or base station, and nothing about a deployment, where record positions do not exist.

D. TRANSFER ACROSS CORPORA

Table 12 and Fig. 6 report the results of transfer from $\mathcal{D}_A$ to $\mathcal{D}_B$ is simply harder”, and it carries a caveat the forward direction does not: publishes no identifiers, so it has no group key, and its own . In terms of macro- F_1 , every supervised model loses 0.10 to 0.13. However, the majority baseline, which uses no features, also loses 0.11, because $\mathcal{D}_B$ has a lower attack prevalence than $\mathcal{D}_A$. Relative to this baseline, the loss in macro- F_1 lies between -0.018 and +0.015, and none of the macro- F_1 losses remains significant after correction. Macro- F_1 is the wrong measure for this comparison. Balanced accuracy, which prevalence does not affect, shows the loss clearly. It decreases from 0.81–0.91 on held-out reference is a random split and therefore an optimistic bound. Its Δ_{F_1} is consequently an over-estimate.

We do not report a fine-tuning result. Few-shot adaptation on labelled target data is a different experiment, it is already published for these corpora [22], and it relocates the problem rather than solving it: it means each deployment requires its own labelled attack data, which is exactly the cost a pre-trained detection xApp was supposed to avoid. Source data to 0.53–0.63 on $\mathcal{D}_B$, a drop of 0.21 to 0.36 that is significant for 5 of the six architectures after correction, is positive in at least 18 of the 20 splits for every architecture, and averages 0.28 [0.15, 0.41]. The trivial baselines remain unchanged, and the target ROC-AUC lies between 0.53 and 0.70.

The degradation is statistically unambiguous. A paired t -test over split seeds rejects the null hypothesis of equal held-out and cross-deployment detectors stay above chance on the target, but not by much. The average target balanced accuracy is 0.57, and the corrected interval excludes 0.5 for 4 of the six architectures (3 after Holm correction). For reference, the stratified sampler achieves a macro- F_1 for all six non-trivial architectures, and every one survives Holm correction across the family ($p_{\text{Holm}} \leq 0.0069$). Effect sizes are large, with d_c between and .

The choice among detector families is a second-order effect beside the shared failure to transfer, and this is the more useful way to read the table: the spread across architectures on the target, of F_1 of 0.424 on $\mathcal{D}_B$, and a fair coin would achieve 0.494. We give both because a macro- F_1 from best to worst, is smaller than the distance any of them has fallen, and F_1 floor depends on the prior it samples from. According to every prevalence-independent measure, LR transfers best (balanced accuracy 0.63, ROC-AUC 0.70), whereas the MLP, which achieves the highest held-out source balanced accuracy (0.91), reaches 0.55 on the target. Source performance does not predict target performance in a prevalence-independent metric: across the six architectures the ordering is not the source ordering. rank correlation in balanced accuracy is -0.26 ($p = 0.62$). In macro- F_1 the rank correlation is 0.83 ($p = 0.04$), but rankings in macro- F_1 change with prevalence, and six architectures provide little precision for either estimate.

A single aggregate score conceals the shape of *Effect of the target labels*. Part of this loss comes from the label conflict of Section VI-C. Detectors that learned to recognize a UDP flood on $\mathcal{D}_A$ detect the floods in capture file 15 and, by the same rule, flag between 0.00 and 0.85 of the benign-labeled copies in file 5, whereas LR detects neither. When the copies are excluded, in a deterministic re-run of 10 of the 20 split seeds, the failure, so Table 14 reports detection per attack category on the target. The benign column is specificity; the remainder are recall. target balanced accuracy is 0.61 to 0.78 (0.74 to 0.78 for the five nonlinear architectures and 0.61 for LR). The loss relative to held-out source data decreases to 0.00 to 0.25 and is significant for 1 of the six architectures after correction. The conflicts reverse the apparent ranking. On all flows LR seems to transfer best, and on the flows with consistent labels it is the worst. Compared with the in-target reference on these flows (Table 10), transfer still falls short by 0.06 to 0.25 in balanced accuracy.

No architecture is usable on both classes. Logistic regression holds benign specificity and detects of the denial-of-service traffic. The MLP detects of it, and of the scanning traffic, while classifying of benign traffic as attack — which is why its macro- F_1 falls beneath the trivial floor. The tree ensembles occupy a middle position and are the weakest at scanning, recovering to of it.

TABLE 11. The 5G-NIDD authors' pipeline [13] reproduced and moved one step per rung: balanced accuracy on $\mathcal{D}_B$. R0 is their protocol. * RF on the test flows without the benign copies of Table 9. KNN is evaluated at R0 only.

Rung	Split	Features	DT	Balanced accuracy				RF	
				RF	KNN	NB	MLP	BA*	FPR
R0	Random 70/30	Published 10	0.9995	0.9996	0.9988	0.9310	0.9986	0.9997	0.0005
R1	Random 70/30	No Seq/Offset	0.7059	0.7059	—	0.6814	0.7042	0.9999	0.5881
R2	Capture file	Published 10	0.8594	0.9226	—	0.7501	0.8669	0.9578	0.0723
R2	Capture file	No Seq/Offset	0.7988	0.7996	—	0.7593	0.8004	0.8958	0.1937
R3	Base station	Published 10	0.7619	0.8949	—	0.6598	0.8577	0.9354	0.0819
R3	Base station	No Seq/Offset	0.6557	0.6555	—	0.6271	0.6919	0.8284	0.3465

The reverse direction, to , is reported separately as a symmetry check rather than as a second headline. It distinguishes “detectors do not transfer” from “

TABLE 12. Transfer from $\mathcal{D}_A$ to $\mathcal{D}_B$ in the 18-column shared space, mean over 20 split seeds. Δ_{BA} : source minus target balanced accuracy, with Nadeau-Bengio corrected 95% interval; *: significant after Holm correction. Last column: macro- F_1 loss minus the majority baseline's loss. Italic rows are the trivial baselines.

Model	Macro- F_1		Src.	Tgt.	Balanced accuracy		Target ROC-AUC	Δ_{F_1} vs. Maj.
	Src.	Tgt.			Tgt.	Δ_{BA} [95% CI]		
LR	0.626	0.530	0.837	0.627	+0.210*	[+0.10, +0.32]	0.701	-0.018
DT	0.619	0.518	0.822	0.533	+0.290*	[+0.08, +0.50]	0.526	-0.013
RF	0.701	0.572	0.808	0.581	+0.227	[-0.01, +0.47]	0.578	+0.015
XGB	0.646	0.532	0.864	0.562	+0.303*	[+0.14, +0.47]	0.572	-0.001
HGB	0.647	0.537	0.867	0.566	+0.301*	[+0.13, +0.47]	0.581	-0.004
MLP	0.655	0.531	0.912	0.553	+0.359*	[+0.22, +0.50]	0.557	+0.010
<i>Strat.</i>	0.494	0.424	0.502	0.500	+0.001	[-0.01, +0.01]	0.500	-0.044
<i>Maj.</i>	0.492	0.378	0.500	0.500	+0.000	[+0.00, +0.00]	0.500	+0.000

This is not a smooth degradation in which every detector gets uniformly worse.

Each architecture fails differently

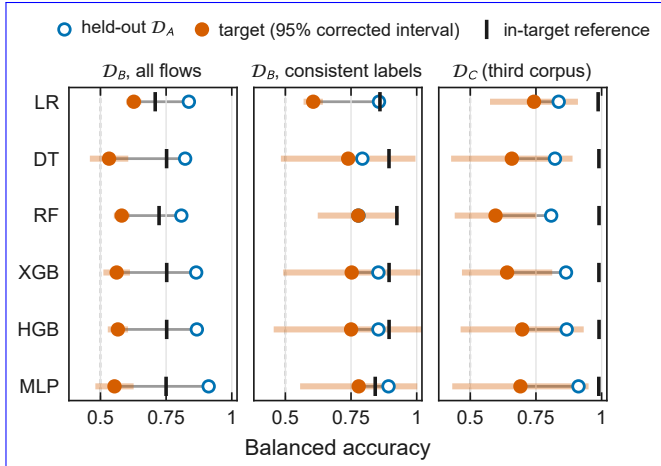


FIGURE 6. Balanced accuracy of detectors trained on $\mathcal{D}_A$: on held-out $\mathcal{D}_A$ (open) and on the target (filled, with the Nadeau-Bengio corrected 95% interval), for $\mathcal{D}_B$ on all flows (20 split seeds), $\mathcal{D}_B$ without the benign copies of Fig. 5 (10 seeds), and the third corpus $\mathcal{D}_C$ (20 seeds). The vertical bar is a reference trained and tested inside the target on a random split. The dashed line is chance, where both input-blind baselines sit. On every target the transferred detectors fall short of the in-target reference.

The reverse direction (Table 13) behaves differently. Balanced accuracy on $\mathcal{D}_A$ reaches 0.61 to 0.76, and the difference is not visible in any single number.

The reverse direction makes the mechanism explicit, change relative to the source reference, which is optimistic because it is based on a random split, ranges from -0.037 to +0.145. LR, XGB, HGB, and the MLP lose little or no balanced accuracy, whereas DT and RF lose a sig-

TABLE 13. Reverse direction, $\mathcal{D}_B$ to $\mathcal{D}_A$. The source reference is a random split of $\mathcal{D}_B$, which has no group key, and is therefore optimistic.

Model	Macro- F_1		Src.	Tgt.	Balanced accuracy		Target ROC-AUC	Δ_{F_1} vs. Maj.
	Src.	Tgt.			Tgt.	Δ_{BA} [95% CI]		
LR	0.708	0.563	0.708	0.745	-0.037*	[-0.05, -0.02]	0.827	+0.254
DT	0.752	0.343	0.751	0.607	+0.145*	[+0.05, +0.24]	0.622	+0.518
RF	0.730	0.337	0.723	0.630	+0.093*	[+0.06, +0.13]	0.845	+0.502
XGB	0.753	0.497	0.752	0.764	-0.012	[-0.03, +0.01]	0.845	+0.364
HGB	0.753	0.488	0.752	0.751	+0.001	[-0.05, +0.05]	0.832	+0.373
MLP	0.752	0.470	0.751	0.728	+0.023	[-0.04, +0.09]	0.814	+0.390
<i>Strat.</i>	0.502	0.417	0.502	0.500	+0.003	[+0.00, +0.00]	0.500	+0.194
<i>Maj.</i>	0.377	0.486	0.500	0.500	+0.000	[+0.00, +0.00]	0.500	+0.000

nificant amount. Macro- F_1 nevertheless decreases sharply, because $\mathcal{D}_A$ contains every category consists of 94.63% attack traffic and a small number of false negatives dominates the small benign class. A model trained on the narrower $\mathcal{D}_B$ does and four more, so nothing is untestable. Categories present in the training corpus transfer: the boosted ensembles recover to of distributed denial of service and to of scanning. Categories absent from it do not. separates the classes of $\mathcal{D}_A$ better than a model trained on $\mathcal{D}_A$ separates those of $\mathcal{D}_B$ contains no. One untested explanation is that the larger and more varied benign traffic of $\mathcal{D}_B$ gives a better model of benign behavior.

Per-category detection (Table 14) shows that no architecture performs acceptably on both classes of $\mathcal{D}_B$. LR keeps a benign specificity of 0.98 but detects only 0.25 of the DoS traffic. The tree models and the MLP detect more attacks and flag 0.43 to 0.56 of the benign traffic. In the reverse direction, categories absent from $\mathcal{D}_B$ are detected unevenly. The ensembles and the MLP recall at most 0.01 of the web attacks, and web recall on is between and for four of six architectures.

A detector does not recognise an attack class it has never been shown, and it fails silently rather than noisily.

whereas brute-force attacks, which are also absent from $\mathcal{D}_B$, are recalled at rates of 0.34 to 0.57.

E. SCORE CALIBRATION AND PRIOR SHIFT COMPOSITION OF THE TRANSFER GAP

All three deployment criteria depend on the score $\hat{p}$ behaving as a probability, because (4) and (7) both operate on a threshold applied to it. We therefore report expected calibration error over fifteen *equal-mass* bins; equal-width bins are misleading here, because scores pile up near 1.0 and most equal-width bins are then empty. **Separability. A classifier trained to distinguish $\mathcal{D}_A$ flows from $\mathcal{D}_B$ flows achieves a balanced accuracy of 0.999 on the 18 shared columns and of 0.997 on the nine exporter-robust columns alone. The largest per-feature shifts are in the protocol mix (most $\mathcal{D}_A$ flows are TCP and most $\mathcal{D}_B$ flows are UDP), the direction ratios, and the packet counts (Fig. 7). The robust columns shift as much as the segmentation-dependent columns ($\bar{W}_1$ of 0.28 compared with 0.27). The corpora thus differ in traffic composition, which no choice of exporter would remove, as well as in flow segmentation, which a common exporter would remove.**

In distribution the models are reasonably calibrated, with ECE between and . Under transfer they are not: target ECE rises to between and . That is **A common exporter. The raw captures of $\mathcal{D}_B$ are public, so we re-extracted them with Zeek 8.0.10, the exporter of $\mathcal{D}_A$, and mapped the Zeek records onto the shared space with the same function as those of $\mathcal{D}_A$. In every capture of $\mathcal{D}_B$, the published attack flows are the traffic of one host pair. Labeling a flow as attack when its host pair is an attack pair of the same capture reproduces 100% of the published labels, and we carried these labels over to the 346,882 Zeek flows (76% attacks). The flood copies of Section VI-C reappear in the Zeek output (66,033 flows), so they lie in the captures and are not an artifact of the Argus export. We then repeated the transfer on the same 10 split seeds as the Argus results without the copies. The trained models are identical, and only the export of the target differs. On all flows, balanced accuracy falls from 0.54–0.62 with Argus to 0.43–0.56 with Zeek, and the paired difference over seeds (–0.15 to –0.06) has a 95% interval below zero for 6 of the six architectures. Without the copies, five architectures change by –0.06 to 0.04, none of them significantly, and LR loses 0.20. The false positive rate rises under Zeek for every architecture, to 0.13–0.46 without the copies against 0.03–0.18 with Argus. A common exporter does not close the gap. What remains is the difference in traffic composition that the expected finding, and it suggested an obvious remedy — calibrate on held-out source data and recover the operating point. We tested it, and the remedy fails in an instructive way.**

Fitting Platt scaling or isotonic regression on a group-disjoint source calibration

fold halves both Brier score and ECE, exactly as advertised, and simultaneously makes the operating point far worse. For logistic regression on the source, Brier falls from to while deployment PPV falls from to and the alert queue grows from to per hour. A calibrator fitted on a corpus that is % attack learns to map scores onto that prior, so applied anywhere it pushes almost every score past $\tau = 0.5$ and the detector flags nearly everything.

There is a stronger statement available, and it explains why no calibrator could have worked. Every method tested here is a monotone transform, so none changes the model's ranking of examples and none changes the set of achievable operating points. We verify this rather than assert it: Platt and temperature scaling preserve ROC-AUC to within 10^{-5} for five of six architectures. **Threshold saturation is therefore not a calibration failure but a discriminability limit. ** Target ROC-AUC lies between and , and no threshold, under any calibration, extracts a usable operating point from a curve that close to the diagonal **domain classifier and the robust columns already indicated.**

One methodological warning follows, because it nearly produced a positive result from nothing. Temperature scaling appears in our sweep to rescue two architectures that raw scores cannot rescue. It preserves the ROC exactly, so it rescues nothing: it rescales the scores so that a *fixed grid* of swept thresholds lands on a different part of the same curve. Any comparison of monotone calibrators over a fixed threshold grid can manufacture this artefact, and **Model families and adaptation. Table 15 evaluates the same splits under three modifications, none of which yields a transferable detector. Restricting the detectors to the ROC should be reported alongside the grid.**

This compounds the earlier findings rather than merely adding to them. The remedy suggested by Fig. ?? — raising the threshold to suppress alert volume — assumes that a high score means high confidence, and under transfer it does not, so the operating point chosen on the source deployment does not carry over even in the ordering it induces.

A second and independent effect is prior shift, and it is severe. Training prevalence on is %, target prevalence on is % **nine exporter-robust columns raises the held-out source balanced accuracy to 0.80 to 0.94, but the target balanced accuracy falls to 0.48 to 0.53, no better than chance. The columns that do not depend on flow segmentation thus carry the shift in traffic composition. CORAL, which uses unlabeled target features, yields a target balanced accuracy of 0.47 to 0.60. Only the MLP benefits from it (0.60 compared with 0.55 without adaptation), and the operational prevalence assumed in Section V is $\pi = 0.002$. A classifier trained under one prior emits scores calibrated to that prior. Where the training prior π_{tr} is known, the scores admit a closed-form correction, which rescales the decision surface without retraining.**

target false positive rate increases to 0.55 to 0.99. The novelty detectors trained on benign traffic alone already fail on held-out source data, because floods and scans resemble short benign flows in this feature space.

We tested this correction and it does not work either, which is worth reporting because it is the obvious next thing to try. Applying (??) requires the target prior, which a deployment does not know; we supply the true value anyway, as an oracle, to bound what the correction could achieve. Even so it does not beat leaving the scores alone: for logistic regression, oracle prior correction gives an operational precision of against for raw scores. It does cut the alert queue sharply, from to per hour, by moving to a far more conservative operating point, and it pays for that with more than half the remaining recall **Cost of the shared space. Projecting $\mathcal{D}_A$ onto the 18 shared columns reduces the in-distribution balanced accuracy by 0.005 to 0.082 relative to the use of all 46 non-port Zeek features (Table 16). The low source macro- F_1 values in Table 12 reflect the**

TABLE 14. Per-category detection, mean over 20 split seeds at $\tau = 0.5$. “Ben.” is specificity; other columns are recall. BF: brute force. Categories absent from $\mathcal{D}_B$ can be tested only in the reverse direction.

Model	$\mathcal{D}_A \rightarrow \mathcal{D}_B$			$\mathcal{D}_B \rightarrow \mathcal{D}_A$ (absent from $\mathcal{D}_B$: DDoS, BF, web)					
	Ben.	DoS	Probe	Ben.	DoS	Probe	DDoS	BF	Web
LR	0.98	0.25	0.64	0.71	0.85	0.92	0.96	0.57	0.30
DT	0.44	0.65	0.29	0.82	0.44	0.47	0.48	0.43	0.13
RF	0.46	0.74	0.19	0.89	0.41	0.55	0.49	0.34	0.00
XGB	0.57	0.58	0.17	0.89	0.73	0.88	0.85	0.35	0.01
HGB	0.54	0.63	0.18	0.87	0.72	0.87	0.83	0.34	0.01
MLP	0.46	0.68	0.25	0.86	0.71	0.73	0.81	0.36	0.00

TABLE 15. Transfer from $\mathcal{D}_A$ to $\mathcal{D}_B$ under alternative feature sets, adaptation, and model families, mean over 20 split seeds. CORAL reads unlabeled target features.

Arm	Model	Src. BA	Tgt. BA	Tgt. AUC	Tgt. FPR
Shared space (18)	LR	0.837	0.627	0.701	0.023
	DT	0.822	0.533	0.526	0.558
	RF	0.808	0.581	0.578	0.541
	XGB	0.864	0.562	0.572	0.429
	HGB	0.867	0.566	0.581	0.463
	MLP	0.912	0.553	0.557	0.539
Exporter-robust subset (9)	LR	0.804	0.510	0.583	0.854
	DT	0.883	0.480	0.488	0.177
	RF	0.894	0.507	0.469	0.206
	XGB	0.925	0.511	0.533	0.089
	HGB	0.939	0.521	0.541	0.090
	MLP	0.928	0.534	0.499	0.039
CORAL (reads target features)	LR	0.836	0.503	0.646	0.902
	DT	0.812	0.500	0.504	0.870
	RF	0.795	0.481	0.515	0.992
	XGB	0.906	0.473	0.478	0.906
	HGB	0.875	0.481	0.484	0.915
	MLP	0.930	0.602	0.632	0.552
Benign-only novelty	IF	0.503	0.492	0.464	0.024
	AE	0.455	0.479	0.484	0.074

TABLE 16. In-distribution cost of the shared space on $\mathcal{D}_A$ flows: all 46 non-port Zeek features against the 18 shared columns, same splits. Loss interval: Nadeau–Bengio corrected, 95%.

Model	BA full	BA shared	Loss [95% CI]	F_1 full	F_1 shared
LR	0.918	0.837	+0.082 [-0.04, +0.20]	0.725	0.626
DT	0.861	0.822	+0.039 [-0.12, +0.20]	0.663	0.619
RF	0.846	0.808	+0.038 [-0.03, +0.11]	0.726	0.701
XGB	0.885	0.864	+0.020 [-0.05, +0.09]	0.677	0.646
HGB	0.893	0.867	+0.026 [-0.11, +0.16]	0.680	0.647
MLP	0.917	0.912	+0.005 [-0.08, +0.10]	0.708	0.655

An earlier version of this study reported a feature-family ablation and drew from it the claim that radio-layer telemetry, which contributes most to in-distribution accuracy

94.63% attack prevalence of $\mathcal{D}_A$ rather than weak detectors, as the corresponding source balanced accuracy is 0.81 to 0.91.

Prior shift is therefore real, and correcting it helps the queue rather than the precision. It is not the binding constraint.

F. ON THE FEATURE-FAMILY ABLATION

F. A THIRD CORPUS

To test whether these results depend on the pair of corpora, we added $\mathcal{D}_C$, the flow records of a 5G core testbed [24], as a second target. It changes the site, the flow exporter, and the vantage point at once,

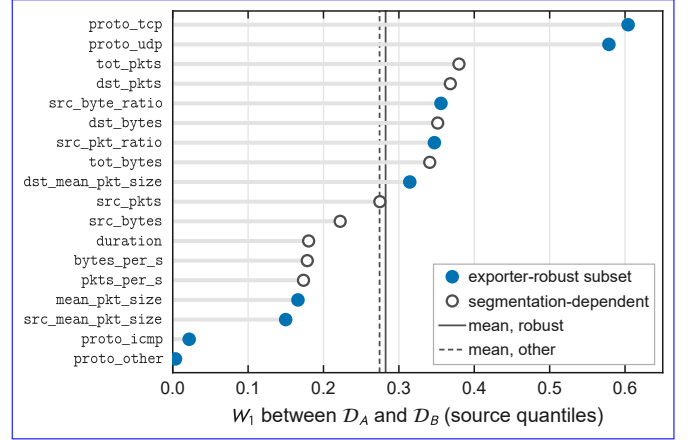


FIGURE 7. Covariate shift between $\mathcal{D}_A$ and $\mathcal{D}_B$ per shared column: first-order Wasserstein distance after the quantile transform fitted on source training data, Eq. (2). Filled markers are the exporter-robust subset, fixed before any result under it was computed; vertical lines are the two group means. The robust columns shift as much as the remaining columns, which indicates that the gap is not solely due to differences in how the two exporters segment flows.

and its labels, unlike those of $\mathcal{D}_B$, are consistent. Only 3.2% of its flows lie on records that also occur with the opposite label, is also the primary source of deployment-specific overfitting, so that removing it improves transfer. and the upper bound on balanced accuracy in the shared columns is 0.992. When trained and tested on $\mathcal{D}_C$ itself, the six architectures achieve 0.987 to 0.990 on a random split and 0.660 to 0.826 when the capture file of one attack type is held out, so that this attack type is not seen during training.

That claim is withdrawn. It was registered in advance with a falsification condition, and the condition fired: the record-level join between the flow records and the radio telemetry cannot be established from the published artefacts, because the flow table carries no time column and shares no identifier with the radio layer. Without that join the ablation cannot be constructed at all, and no result we could report here would be supported by the data. For transfer from $\mathcal{D}_A$ (Table 17, 20 split seeds), the balanced accuracy on $\mathcal{D}_C$ ranges from 0.60 to 0.74, and ROC-AUC ranges from 0.70 to 0.93. Every architecture detects the SYN flood, an attack type that is represented in training. Recall on the ICMP flood ranges from 0.10 to 1.00, and recall on the control-plane PFCP attack ranges from 0.00 to 0.99. The main source of error is the benign traffic

of the new site. The false positive rate ranges from 0.225 to 0.807, and up to 0.81 of the signalling traffic of the core network is flagged. The loss relative to held-out source data, 0.09 to 0.22, varies so much between split seeds that none is significant after correction. For transfer from $\mathcal{D}_B$ (10 split seeds), balanced accuracy spans 0.42 to 0.85, and ROC-AUC is as low as 0.38. Which architecture transfers best depends on the pair of corpora, and no architecture transfers consistently. At $\pi = 0.002$, the highest operational precision at any threshold that retains recall above 10% is 0.092 for transfer from $\mathcal{D}_A$ and 0.031 for transfer from $\mathcal{D}_B$, and no architecture passes the deployability test in either direction (0 and 0 of six). The pattern of Section VI-D recurs at a third site with a third exporter and consistent labels. The SYN flood is detected, and much of the site's benign traffic is not recognized as benign.

We record the withdrawal rather than deleting the question, because the question remains a good one and because the pre-registered failure criterion is what makes the withdrawal reportable rather than embarrassing. Fard *et al.* publish the in-distribution half of the effect on this same corpus; the transfer half, which is the part that would have been ours, is untested.

G. ALERT BURDEN

Fig. ?? sweeps the decision threshold and reports both the resulting hourly alert volume from (3) and the operational precision from (4).

The gap between the two precision figures is **Table 18 reports pooled operating points at $\tau = 0.5$ for three evaluation sets, together with 95% cluster-bootstrap intervals. On the radio layer, the result. On the corpus, precision at six detectors have a pooled window-level false positive rate of 0.22 to 0.33 (50 to 74 false alerts per benign UE-hour) and, at $\pi = 0.002$, an operational precision of 0.0055 to 0.0080, compared with a corpus precision of 0.90 to 0.93. These point estimates are not stable. The false positive rate depends on which of the ten benign sessions are included in the test set (Table 7), and the bootstrap interval of the pooled rate spans 0.01 to 0.65 across architectures, or 3 to 146 false alerts per benign UE-hour. Likewise, the interval for operational precision at $\tau = 0.5$ is approximately for every detector, and the six are indistinguishable from one another. Translated through (4) at $\pi = 0.002$, pooled operational precision is between and — a collapse by a factor of , with the detectors still indistinguishable. Alert volume runs to tens of thousands per hour, spans 0.0028 to 0.13. Thirty sessions cannot pin down the alert burden of the radio layer. They show only that it depends on the large majority false.**

Two properties of this collapse matter more than its size at any single operating point. First, it is not an artefact of the base rate we declared. Sweeping π from to moves the collapse factor from to , and the detectors remain within a factor of one another throughout. Second, **benign conditions a detector meets. On held-out $\mathcal{D}_A$ flows, the two-dimensional parameter space is really one-dimensional: operational precision depends on π and is invariant to the benign arrival rate, while alert volume scales linearly in that rate and independently of it. We verify both numerically rather than asserting them.**

false positive rate is 0.066 to 0.201, which at $\lambda_b = 240,000$ benign flows per hour corresponds to 15,726 to 48,289 false alerts per hour. On $\mathcal{D}_B$,

the false positive rate is 0.023 to 0.558, or 5,620 to 133,990 false alerts per hour. Without the benign copies described in Section VI-C, it is 0.026 to 0.183, which still amounts to 6,158 to 44,003 false alerts per hour, and the operational precision is 0.0072 to 0.0398.

A caution on how this quantity is estimated. Operational precision is strongly non-linear. Increasing the decision threshold improves precision only on held-out $\mathcal{D}_A$ flows (Fig. 8). The best threshold is chosen on the evaluation data, so the following values are upper bounds. With recall kept above 10%, high thresholds reach an operational precision of 0.42 on held-out $\mathcal{D}_A$ flows, whereas the best value achieved by any detector on $\mathcal{D}_B$ is 0.026, or 0.063 without the copies. On the radio layer, the best pooled value is 0.036, but its bootstrap interval reaches 1.0 for 4 of the six architectures. A resample that omits the benign sessions flagged almost entirely leaves thresholds without any false positive. The relation between corpus precision and operational precision follows arithmetically from π . Varying π from 10^{-4} to 0.5 changes the ratio between the two from 2,627 to 1.18 (Fig. 9), so an operational precision is meaningful only when the assumed prevalence is stated. **Estimation of operational precision. PPV is strongly nonlinear** in the false positive rate near zero, so a single evaluation fold that happens to produce no false positives yields a precision of exactly one. Averaging that across folds, rather than pooling confusion counts, inflates the apparent spread between architectures: on our radio folds, which carry a median of benign windows, the same data gives a spread of pooled and **contain no false positive yields PPV = 1 regardless of its recall. Averaging PPV over folds favors the architecture that happens to draw the most such folds. On the radio layer, the same counts yield a ratio between the best and the worst architecture of 1.45 when pooled, 1.89 as a median over folds, and 16.0 as a mean over folds . The distortion tracks the number of negatives per fold — it is on the much larger network folds and vanishes when the target corpus is evaluated whole. We report (Table 19). We use pooled counts throughout, and report the mean alongside so that the size of the artefact remains visible. An earlier version of this work reported the mean. •**

The cross-deployment case is considerably worse, and it separates the architectures in a way the source corpus does not. On at $\tau = 0.5$ the false positive rate ranges from for logistic regression to for the MLP. At the same benign flow rate that is false alerts per hour at one end and at the other, against pooled operational precisions of and respectively — a factor of in burden for of

H. CALIBRATION AND PRIOR SHIFT

Under transfer, the scores are poorly calibrated (Fig. 10). We fitted three calibrators on a group-disjoint calibration fold drawn from the source (Table 20). Whether a calibrator can alter the achievable operating points depends on whether it preserves the order of the scores. Temperature scaling preserves this order and changes ROC-AUC by at most $2e-05$. Platt scaling is monotone, but its fitted slope

TABLE 17. Transfer to the third corpus $\mathcal{D}_C$ (5G core testbed, NFStream flows [24]): balanced accuracy on held-out source data and on $\mathcal{D}_C$, target ROC-AUC, recall per attack type, and the share of the core's background signalling flagged (Bkg.). Mean over 20 split seeds from $\mathcal{D}_A$ and 10 from $\mathcal{D}_B$, at $\tau = 0.5$.

Model	Src. BA	Tgt. BA	Trained on $\mathcal{D}_A$					Trained on $\mathcal{D}_B$	
			AUC	SYN	ICMP	PFCP	Bkg.	Tgt. BA	AUC
LR	0.837	0.742	0.800	1.00	0.10	0.00	0.27	0.517	0.726
DT	0.822	0.658	0.696	1.00	1.00	0.79	0.67	0.422	0.384
RF	0.808	0.596	0.926	1.00	1.00	0.99	0.81	0.540	0.697
XGB	0.864	0.640	0.888	1.00	1.00	0.74	0.70	0.688	0.814
HGB	0.867	0.698	0.932	1.00	1.00	0.80	0.61	0.565	0.632
MLP	0.912	0.691	0.837	1.00	0.20	0.61	0.44	0.848	0.766
<i>Maj.</i>	0.500	0.500	0.500	1.00	1.00	1.00	1.00	0.500	0.500

TABLE 18. Pooled operating points at $\tau = 0.5$, 20 split seeds. FPR interval: 95% cluster bootstrap resampling split seeds and clusters (radio sessions, $\mathcal{D}_A$ source addresses, $\mathcal{D}_B$ capture files; for flows from a deterministic re-run on 10 seeds, EXP-056, on which the “w/o conflicts” block is also computed). “w/o conflicts”: $\mathcal{D}_B$ without the benign flows whose record also occurs with an attack label. PPV at $\pi = 0.002$. [†]False alerts per benign UE-hour (radio, 225 windows per UE-hour) or per hour at $\lambda_b = 240,000$ benign flows (flows). [‡]Highest pooled PPV over all thresholds with recall at least 0.10.

Corpus	Model	TPR	FPR [95% CI]	Corpus P	PPV	False alerts [†]	Best PPV [‡]
$\mathcal{D}_A$ radio	LR	0.798	0.223 [0.017, 0.547]	0.921	0.0071	50	0.0130
	DT	0.910	0.329 [0.086, 0.639]	0.900	0.0055	74	0.0055
	RF	0.954	0.267 [0.024, 0.650]	0.921	0.0071	60	0.0360
	XGB	0.931	0.251 [0.013, 0.606]	0.924	0.0074	57	0.0116
	HGB	0.945	0.262 [0.018, 0.615]	0.922	0.0072	59	0.0086
	MLP	0.895	0.222 [0.019, 0.560]	0.929	0.0080	50	0.0109
	<i>Maj.</i>	1.000	1.000 [1.000, 1.000]	0.766	0.0020	225	0.0020
$\mathcal{D}_A$ network	LR	0.892	0.201 [0.126, 0.445]	0.992	0.0088	48 289	0.0297
	DT	0.852	0.157 [0.068, 0.498]	0.994	0.0108	37 658	0.0120
	RF	0.920	0.198 [0.121, 0.611]	0.993	0.0092	47 568	0.4172
	XGB	0.884	0.090 [0.027, 0.373]	0.997	0.0193	21 625	0.2735
	HGB	0.881	0.066 [0.031, 0.112]	0.998	0.0262	15 726	0.2727
	MLP	0.905	0.071 [0.043, 0.349]	0.997	0.0250	16 996	0.2189
	<i>Maj.</i>	1.000	1.000 [1.000, 1.000]	0.967	0.0020	240 000	0.0020
$\mathcal{D}_B$ (transfer)	LR	0.277	0.023 [0.009, 0.079]	0.948	0.0232	5 620	0.0258
	DT	0.624	0.558 [0.111, 0.783]	0.633	0.0022	133 990	0.0022
	RF	0.703	0.541 [0.121, 0.796]	0.667	0.0026	129 843	0.0027
	XGB	0.552	0.429 [0.013, 0.687]	0.666	0.0026	102 890	0.0026
	HGB	0.596	0.463 [0.031, 0.699]	0.665	0.0026	111 104	0.0026
	MLP	0.646	0.539 [0.030, 0.745]	0.649	0.0024	129 320	0.0026
	<i>Maj.</i>	1.000	1.000 [1.000, 1.000]	0.607	0.0020	240 000	0.0020
$\mathcal{D}_B$ w/o conflicts	LR	0.269	0.057 [0.038, 0.086]	0.947	0.0094	13 629	0.0102
	DT	0.662	0.183 [0.103, 0.294]	0.931	0.0072	44 003	0.0077
	RF	0.719	0.165 [0.118, 0.258]	0.943	0.0087	39 561	0.0629
	XGB	0.531	0.026 [0.012, 0.047]	0.987	0.0398	6 158	0.0631
	HGB	0.548	0.047 [0.030, 0.080]	0.978	0.0227	11 315	0.0587
	MLP	0.612	0.053 [0.026, 0.102]	0.977	0.0225	12 794	0.0306
	<i>Maj.</i>	1.000	1.000 [1.000, 1.000]	0.790	0.0020	240 000	0.0020

TABLE 19. Operational precision on the radio layer at $\tau = 0.5$, $\pi = 0.002$, estimated three ways from the same 20 folds.

Model	Folds with FP = 0	Pooled	Median	Mean
LR	1	0.0071	0.0118	0.0815
DT	0	0.0055	0.0070	0.0137
RF	1	0.0071	0.0092	0.0814
XGB	4	0.0074	0.0095	0.2185
HGB	2	0.0072	0.0091	0.1291
MLP	0	0.0080	0.0132	0.0476

can be negative, and for DT it inverted the ranking in 1 of 10 seeds, changing ROC-AUC by 0.95. In that seed, the scores of the tree carried no ranking information on the calibration fold (ROC-AUC 0.48

on 96 held-out source addresses), although they ranked the test fold well (0.97). Platt scaling fitted the calibration fold correctly with a slope of -0.55 and, as a result, inverted the ranking on the test fold. The fault lies with small group-disjoint calibration folds, which need not resemble the test fold, and not with Platt scaling itself. Isotonic regression is monotone but piecewise constant, so it merges scores into ties and, on its fitting data, yields the ROC convex hull [54]. Its largest change in ROC-AUC ranged from 0.04 to 0.21 across architectures. None of the calibrators raises the target macro- F_1 . A transferred detector does not degrade gracefully into a noisier detector; it degrades into an alert generator that an operator will switch off within a shift F_1 above that of

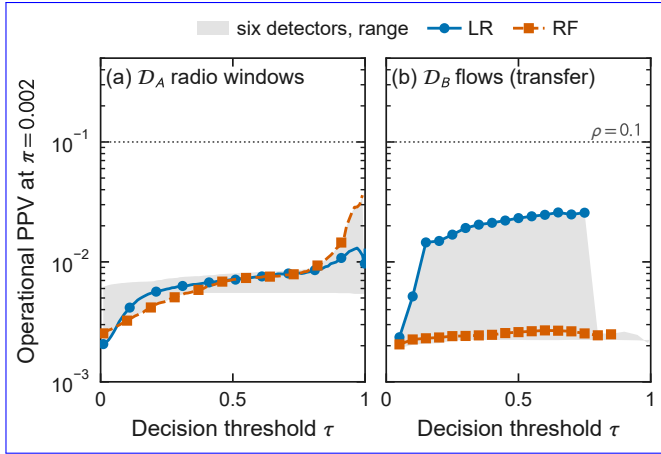


FIGURE 8. Pooled operational precision at $\pi = 0.002$ against the decision threshold on (a) the $\mathcal{D}_A$ radio layer and (b) $\mathcal{D}_B$ flows after transfer, at thresholds that keep recall at or above 0.10. The gray band spans the six architectures, and the dotted line marks the precision bound $\rho = 0.1$ of the deployability test, which is not reached at any threshold on either evaluation set.

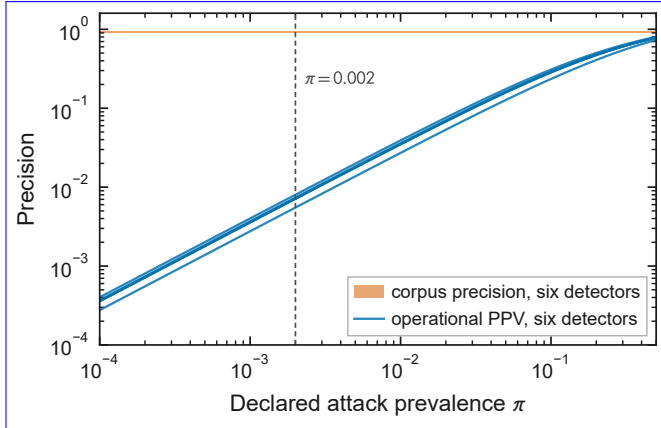


FIGURE 9. Pooled operational precision at $\tau = 0.5$ against the declared attack prevalence π for the six radio-layer detectors (one line each), with their corpus precision as a band. The dashed rule marks $\pi = 0.002$. Operational precision is computed from the same confusion counts with Eq. (4), and only the declared prevalence is varied.

the raw scores for any architecture. Because they are fitted on a source containing 94.63% attacks, they shift the scores toward that prior and flag more benign traffic on $\mathcal{D}_B$.

Two points deserve emphasis because they run against the intuition the source corpus builds. First, The standard remedy for prior shift is to rescale the scores to the target prior [50], which a deployment does not know. Expectation maximization [50] and black-box shift estimation [51] estimate this prior from unlabeled data, assuming that only the class balance changes. Here that assumption fails. For a true target prior of 0.607, the mean estimates per architecture range from 0.16 to 0.86, and 41% of the detectors are operationally indistinguishable on held-out source data — pooled PPV spans a factor of — and an order of magnitude apart on the target, where the spread is . Which corpus one evaluates on decides whether the architectures look alike or not . Second, the model that wins Individual estimates fall below 0.05 or above 0.95. Rescaling with the true prior, an oracle setting, does not beat the raw scores either (Table 20). What limits the detectors on the

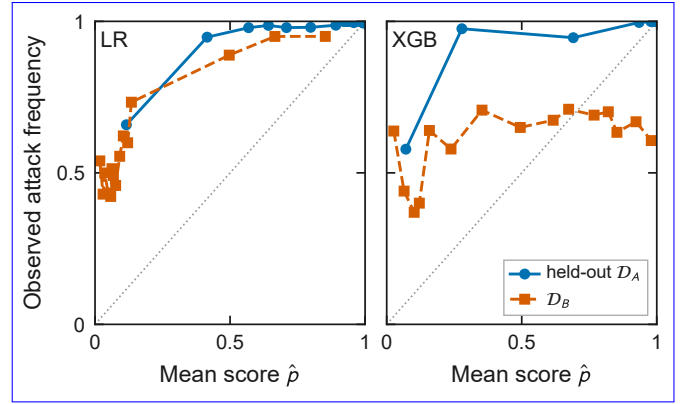


FIGURE 10. Reliability diagrams for LR and XGB on held-out $\mathcal{D}_A$ and on $\mathcal{D}_B$, fifteen equal-mass bins pooled over 20 split seeds.

target operationally is the simplest one, and it is not the model that wins on macro- F_1 there.

Whichever single metric is chosen, it recommends a different system.

is discrimination (ROC-AUC 0.53 to 0.70), not calibration.

I. LATENCY AND RESOURCE FOOTPRINT

I. DECISION LATENCY

Table 21 reports the latency decomposition of (5) at an offered load of 1,000 decisions per second, together with sustainable throughput under the p99 constraint of (??) and container resource consumption. Fig. 11 shows how the per-call latency on one host for three implementations of each model. The first calls scikit-learn with a one-row pandas DataFrame, as our experiment code does. The second calls scikit-learn with a NumPy array on a single thread. The third uses ONNX Runtime on a single thread and is applied only where the exported model reproduces the decisions of scikit-learn (12 of 12 exports, with a decision agreement of at least 1.0000). A no-op call has a 99th percentile grows with load.

Three observations follow. First, median latency is a misleading summary. Four of the six architectures sit inside a 10percentile of 0.001 ms budget at p50 and stay inside it at p99; the two that do not — histogram gradient boosting and the random forest — miss it by an order of magnitude, with p99 of and ms against medians of and ms. The tail, not the median, decides conformance, so platform overhead does not affect the comparisons.

Second, the verdict depends on where the budget is drawn, and O-RAN specifies the near-real-time loop over a range spanning two orders of magnitude. At 1 the implementation has a larger effect on latency than the architecture (Fig. 11). For LR, the DataFrame path takes 0.44 ms nothing conforms; at 5 ms the decision tree alone; at 10 ms the tree, logistic regression, XGBoost and at the MLP; at 100ms against 0.11 ms all but the random forest; at if for an array, and input validation accounts for the difference. For RF, predicting one row through the default thread pool takes 29 s everything. Asserting a single budget converts an authoring choice into an apparent measurement, so we report the crossing point per architecture instead.

Third, and least expected, inference is not the bottleneck. Packet-level feature extraction costs a median ms per flow against to at the median, compared with 5.6 ms of inference for the four architectureslight enough to meet a 100 on one thread and 0.012 ms budget. We re-implemented the exporter with bulk array extraction and measured a speed-up of to times, verified to produce identical flow records; extraction still costs in ONNX Runtime. Across architectures, the 99th percentile for ONNX Runtime lies between 0.014 and 0.035 ms per flow at best. Set against a

TABLE 20. Calibration over 10 split seeds. Left: largest absolute change in ROC-AUC caused by each calibrator, over seeds and both evaluation sets. Right: target macro- F_1 at $\tau = 0.5$ for raw scores, Platt and isotonic calibration, and Platt followed by prior correction with the EM estimate or the true (oracle) target prior.

Model	max $ \Delta \text{ROC-AUC} $			Target macro- F_1 at $\tau = 0.5$				
	Temp.	Platt	Iso.	Raw	Platt	Iso.	+EM	+Oracle
LR	$< 10^{-6}$	1×10^{-6}	0.209	0.538	0.415	0.391	0.327	0.478
DT	2×10^{-5}	0.946	0.170	0.538	0.395	0.405	0.447	0.436
RF	$< 10^{-6}$	$< 10^{-6}$	0.050	0.579	0.504	0.470	0.440	0.488
XGB	$< 10^{-6}$	$< 10^{-6}$	0.150	0.555	0.450	0.394	0.456	0.518
HGB	$< 10^{-6}$	$< 10^{-6}$	0.095	0.549	0.429	0.399	0.479	0.507
MLP	2×10^{-5}	2×10^{-5}	0.045	0.578	0.405	0.408	0.475	0.534

deployed implementation rather than a prototype — on the radio layer, whereas it reaches up to 21.5 μs ms for scikit-learn with arrays. Ranking architectures by latency largely ranks implementations, and a comparison with measurements from other hosts or languages [23] says nothing about the architectures themselves. For context, Obiwewwi et al. report inference times in the microsecond range for logistic regression inside a real Near-RT and a small MLP inside an OpenAirInterface and FlexRIC near-real-time RIC [23]—extraction is not merely the larger half of, within an order of magnitude of our ONNX Runtime figures.

For a window-level detector, the decision path consists of window aggregation followed by inference. Aggregating one 16-record window requires 0.064 ms at the budget but essentially the whole of it. Optimisation effort directed at model compression addresses the smaller part of the problem: 99th percentile, and the ranking of architectures by prototype latency is a statement about the prototype's toolchain rather than about the architectures' upper bound of the 99th percentile of aggregation plus ONNX inference is at most 0.12 ms across the architectures with a verified export. On this host, these two terms alone would meet any budget in the specified range. A repetition of the radio measurement on the same host, in which every call was recorded for Fig. 11, yielded medians 1.02 to 2.1 times those in Table 21 (median ratio 1.46). Per-call latency on a host without CPU isolation thus depends on the state of the host as well as on the model. Nevertheless, the upper 95% bound of the 99th percentile of aggregation plus ONNX inference remained at most 0.36 ms in the repeated measurement.

Table 22 adds the remaining terms of Eq. (5), measured in the FlexRIC loop of Section V. We ran the six models with a report every 10 ms (5,000 reports each after 200 warm-up reports) and LR and RF with a report every 1 ms (20,016 reports each after 1,000 warm-up reports). A report carried 7.7 UEs on average, and the xApp scored every one of them. At the 10 ms period, a report needs 1.64 ms at the median to travel from the E2 node to the xApp, and the control round trip takes 0.78 ms. Inference in the same loop takes at most 0.229 ms at the 99th

percentile. Delivery and control make up at least 86% of the mean loop time for every model, so the choice of model has little effect once a RIC is in the path. The whole loop, from the report to the acknowledged control message, takes 4.45 to 4.95 ms at the 99th percentile, with an upper 95% bound of at most 5.01 ms. It meets the smallest specified budget of 10 ms at the 99.9th percentile as well (at most 5.71 ms), and the largest single value at this period was 11.2 ms.

The loop ran faster at the 1 ms period. Its 99th percentile fell to 1.42 to 1.69 ms, the xApp never held more than 4 reports in its queue, and no report was dropped in any run. Every term became faster, inference included (at most 0.050 ms at the 99th percentile). We did not control CPU power management, and a host that idles between sparse reports and pays for waking up would produce this pattern. The loop runs on one host, with an emulated node and loopback SCTP. It therefore bounds the processing cost of the RIC software path, but it does not include the transport delay of a deployment.

For a flow-level detector, feature construction is more costly than inference. On the three DDoS captures of $\mathcal{D}_A$, the vectorized exporter produces flow records at 0.20 to 0.90 ms per flow, which is 22.8 to 41.6 times faster than the per-packet reference implementation on the same captures (4.49 to 37.34 ms per flow), and the cost per flow increases with the number of packets per flow. On the 2 captures that were compared record by record, the two exporters agree exactly in packets and bytes on all 38,032 shared IPv4 flow keys. The reference implementation additionally emits 22 and 41 IPv6 link-local keys, which the vectorized exporter does not parse because it supports IPv4 only. Mapping one exported record onto the shared space with the pandas code of our pipeline requires an additional 8.5 ms at the median, whereas ONNX inference on flow features requires at most 0.037 ms at the 99th percentile. These figures describe a Python pipeline on a single host. They indicate that the cost lies mainly in feature construction, but they do not indi-

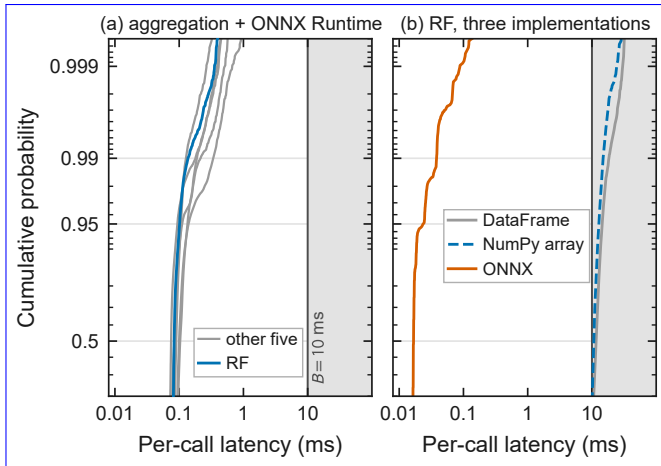


FIGURE 11. Empirical distribution of per-call latency for the radio layer on one host, 20,000 calls per stage, on a logit scale so that the median, 95th, and 99th percentiles are all visible. (a) The decision path of the deployability test, window aggregation followed by ONNX Runtime, for the six architectures. (b) One architecture, RF, under the three implementations of Table 21. The shaded region lies beyond $B = 10$ ms, the smallest budget in the specified range. Only feature construction and inference are timed here. Table 22 gives the full loop through a RIC. The curves come from a repeat of the measurement that kept every call (EXP-060); Table 21 reports the original run.

cate what a native exporter at the user plane would cost.

VII. MEASURED RESULTS

This section reports measurements taken on the radio-telemetry layer of under the run-disjoint protocol of Section V. Unless stated otherwise every figure is a mean over split seeds, with intervals given as paired t -intervals over split means. Model seeds are nested inside split seeds and averaged before testing, so the effective sample size is the number of splits, not the number of fits.

A. EVALUATION PROTOCOL EFFECTS

We first quantify how much of a reported score is attributable to the split protocol rather than to the detector. The same models, features and seeds are evaluated twice: once under a random split over windows, the convention in much of the intrusion-detection literature, and once under a split in which whole capture runs are held out.

Table ?? and Fig. ?? show the result. Every non-trivial architecture scores materially higher under the random split, and all six differences survive Holm correction. The effect is not small: it ranges from to points of macro- F_1 , with paired effect sizes d_z between and .

Two features of the pattern matter more than its magnitude. First, the ordering tracks model flexibility. A single depth-12 decision tree gains most, the boosted ensembles next, the random forest and the multilayer perceptron less, and logistic regression least. Second, the two trivial baselines gain nothing: the majority-class classifier shows $\Delta = -0.002$ with $p = 0.40$. A classifier with no capacity to memorise cannot profit from a leaky split, which is what identifies the mechanism as memorisation rather than as a change in task difficulty.

The practical consequence is that the inflation cannot be treated as a constant offset. It is largest for precisely the flexible architectures that are most often reported, so a reader cannot mentally discount a published random-split figure by a fixed amount.

VII. DISCUSSION

A. IMPLICATIONS FOR O-RAN PRACTICE

B. OPERATIONAL ALERT BURDEN

Data path. E2SM-KPM delivers aggregated measurements for each reporting period and does not carry packets. A window-level detector can operate inside the RIC on KPM reports, whereas a flow-level detector requires a flow exporter located near the user plane, either in the central unit or at the user-plane function, as well as a means of transporting its records to the xApp, for example through a custom E2 service model or the non-real-time data collection path. This transport adds to t_{ind} in Eq. (5). Our RIC loop measures t_{ind} for KPM reports only, so it does not cover the flow layer.

Criterion 2 asks what the detector costs an operator rather than how it scores on a corpus. Table ?? reports both quantities side by side at the default threshold, under the declared deployment parameters $\pi = 0.002$ and $\lambda_b = 240,000$ benign flows per hour. **Detection delay.** Computational latency accounts for only a small part of the total time to detection. A window-level decision cannot be made before the end of its window (16 s in this study), and a flow-level decision cannot be made before the flow record is exported, which, for a long connection, requires an idle or active timeout to expire. The KPM reporting period and the flow timeouts should be reported together with the computational latency.

The two precision columns are the criterion in one line. Corpus precision sits between and for all five detectors, a spread of . Once a realistic base rate is applied, the operational precision of the same five spans to , a factor of six. The metric a paper would report cannot distinguish these detectors; the metric an operator lives with separates them sharply. Absolute volumes are to alerts per hour. **Acting on alerts.** An alert that triggers an E2SM-RC action, such as isolating or steering a UE, acts on a false alert in the same way as on a true one. At the large majority of them false, false positive rates reported in Table 18, automatic mitigation on the radio layer would act against each benign UE 50 to 74 times per hour. Alerts of this quality may inform an operator or a non-real-time policy through A1, but they should not trigger automatic control actions.

None of this is a new statistical result. The base-rate argument is Axelsson's, and it is a quarter of a century old.

What is new is its quantification for a specific detector under a protocol that holds capture runs out, reported alongside the tail-latency measurement for the same model artefacts.

A second and less expected finding concerns the operator's only runtime control. Sweeping the decision threshold to recover precision, Table ?? shows that $\text{PPV} = 0.5$ is *unreachable at any threshold* for four of the five detectors. The scores of the boosted models and the perceptron saturate near unity under a run-disjoint split, so raising τ barely moves the alert volume: the histogram-boosted ensemble still emits alerts per hour at $\tau = 0.99$. Only the random forest and the linear model reach $\text{PPV} = 0.5$, and both do so by collapsing recall, to and respectively. This is a calibration failure with an immediate operational consequence, and it is invisible in accuracy, F_1 or AUC. **Admission and monitoring.** The deployability test of Eq. (6) can serve as an admission check before an xApp is enabled at a new site, provided that labeled benign and attack traffic is available

TABLE 21. Per-call latency in milliseconds on one host (Section V): 20,000 calls after 1,000 warm-up calls, one thread per library, garbage collection enabled. No RIC in the path (Table 22 gives the loop through a RIC). Radio path $\bar{p}_{99}$: upper 95% bound of the 99th percentile of window aggregation plus ONNX inference. Dashes: no export that reproduced scikit-learn's decisions.

Model	sklearn, DataFrame		sklearn, array		ONNX Runtime		Radio path	Flow layer, p_{99}	
	p_{50}	p_{99}	p_{50}	p_{99}	p_{50}	p_{99}	$\bar{p}_{99}$	array	ONNX
LR	0.44	0.72	0.11	0.17	0.010	0.023	0.12	0.16	0.025
DT	0.31	0.52	0.03	0.04	0.010	0.014	0.11	0.06	0.024
RF	6.25	12.99	5.57	21.50	0.012	0.024	0.10	4.45	0.029
XGB	0.73	1.00	0.16	0.32	0.011	0.021	0.09	0.40	0.024
HGB	1.48	2.72	1.17	1.92	0.013	0.035	0.12	1.22	0.034
MLP	0.43	0.67	0.11	0.21	0.017	0.035	0.11	0.15	0.037

TABLE 22. Decision loop through a FlexRIC near-real-time RIC with an emulated E2 node on one host, in milliseconds, as ranges over models, at a reporting period of 10 ms or 1 ms. Terms follow Eq. (5): delivery from the E2 node to the xApp (t_{ind}), queueing in the xApp (t_q), window aggregation (t_{feat}) and ONNX Runtime inference (t_{inf}) for one UE, and the control message up to its acknowledgment (t_{act}). "Loop" runs from the timestamp in the KPM header to the acknowledgment, for the first UE of a report or for all its UEs.

Term	10 ms, six models		1 ms, LR and RF	
	p_{50}	p_{99}	p_{50}	p_{99}
t_{ind}	1.51–1.86	3.02–3.05	0.51–0.53	0.84–0.98
t_q	0.063–0.071	0.18–0.21	0.015–0.016	0.069
t_{feat}	0.001	0.002–0.003	<0.001	0.001
t_{inf}	0.050–0.12	0.13–0.23	0.014–0.025	0.038–0.050
t_{act}	0.72–0.87	1.40–1.50	0.20–0.21	0.53–0.60
Loop, 1 UE	2.38–2.97	4.36–4.59	0.76–0.79	1.38–1.59
Loop, all	2.48–3.18	4.45–4.95	0.77–0.86	1.42–1.69

from that site. After admission, unsupervised shift detection [55] can indicate when the incoming traffic departs from the training distribution. Because the corpora studied here can be distinguished with a balanced accuracy of 0.999, such a monitor would raise an alarm immediately.

B. APPLYING THE DEPLOYABILITY TEST

C. DECISION LATENCY AGAINST A SWEPT BUDGET

We instantiate Eq. (6) with the bounds described below. These bounds were chosen by us, since no O-RAN specifies the near-real-time control loop over a range of specification defines them, and an operator would set them according to its own traffic and staffing. The generalization bound is $\delta = 0.10$ in balanced accuracy and is applied to the upper end of the corrected 95% interval. The alert term requires a threshold that detects at least half of the attacks ($r_{min} = 0.5$), for which at least one alert in ten is a true alert ($\rho = 0.1$ at $\pi = 0.002$), and which raises at most $A_{max} = 200$ false alerts per hour at $\lambda_b = 240,000$ benign flows per hour. All alert conditions are evaluated on $\mathcal{D}_B$, because a detector is deployed on the target rather than on the source. The latency term uses the strictest budget permitted by the specification, $B = 10$ ms. We evaluate it on the FlexRIC loop at a 10 ms to 1 s rather than at a single value. Reporting conformance against one chosen point inside that range makes the verdict an artefact of the choice. Table ?? and Fig. ?? therefore report, for each

architecture, the smallest budget its ms reporting period. There the upper bound of the 99th percentile conforms to.

The floor experiment was run first. A no-op model on the same path yields $p_{99} = 0.103$ percentile is at most 5.01 ms, two orders of magnitude below the tightest budget considered, so the ordering below is a property of the models rather than of the measurement platform.

The verdict moves with the budget. At ms no architecture conforms. At ms only the single decision tree does. At ms four of six conform.

At ms all of them do. One set of measurements thus supports four different conclusions, separated only by where the line is drawn, which is ms for every architecture (Section VI-I), so the term is satisfied for any budget in the specified range, on one host and with an emulated E2 node.

Table 23 presents the result, according to which none of the architectures passes the test (0 of six). Every architecture fails the generalization term, with upper bounds of 0.32 to 0.50, and every architecture also fails the alert term by a wide margin. At a recall of at least 0.5, the highest operational precision on $\mathcal{D}_B$ is 0.0027, and the smallest number of false alerts per hour is 95,507. The verdict is therefore robust to the choice of bounds. It holds for any ρ above 0.0027, any A_{max} below 95,507 per hour, and any δ below 0.32, and it does not depend on the latency term. Nor does it depend on the declared prevalence. The number of false alerts does not involve π and exceeds A_{max} at every threshold, and the precision condition alone would require $\pi \geq 0.076$, which is 38 times the declared value. The verdict is also unaffected by the label conflicts described in Section VI-C. When the test is evaluated on $\mathcal{D}_B$ without the benign copies, 0 of six architectures pass, the highest operational precision at a recall of at least 0.5 is 0.0419, and the reason to report the crossing point instead of smallest number of false alerts per hour is 5,603. On these flows the precision condition alone would hold from $\pi = 0.005$, but the number of false alerts still excludes every architecture. Fig. 12 shows how far the bounds would have to be relaxed. An architecture satisfies the alert term only if an operator accepts several thousand false alerts per hour and a pass-fail column. precision of a few percent, and it satisfies the generalization term only if δ exceeds 0.32. The latency term would matter only if the first two were satisfied, and in our loop it is

TABLE 23. Deployability test of Eq. (6) per architecture. $\bar{\Delta}_{BA}$: upper 95% bound of the balanced-accuracy loss (pass if ≤ 0.10). Best PPV and minimum false alerts per hour over thresholds with recall ≥ 0.5 on $\mathcal{D}_B$ (pass if PPV ≥ 0.1 and alerts ≤ 200). $B_{\min}$: smallest budget of the series 1, 2, 5, 10 ms met by the upper 95% bound of the 99th percentile of the FlexRIC loop (all UEs of a report, 10 ms period, emulated E2 node).

Model	$\bar{\Delta}_{BA}$	Gen.	Best PPV	Min. alerts/h	Alert	$B_{\min}$ (ms)	$\mathcal{P}$
LR	0.32	×	0.0024	166 328	×	5	0
DT	0.50	×	0.0022	118 342	×	5	0
RF	0.47	×	0.0027	98 410	×	5	0
XGB	0.47	×	0.0026	95 507	×	5	0
HGB	0.47	×	0.0026	99 277	×	10	0
MLP	0.50	×	0.0026	105 744	×	5	0

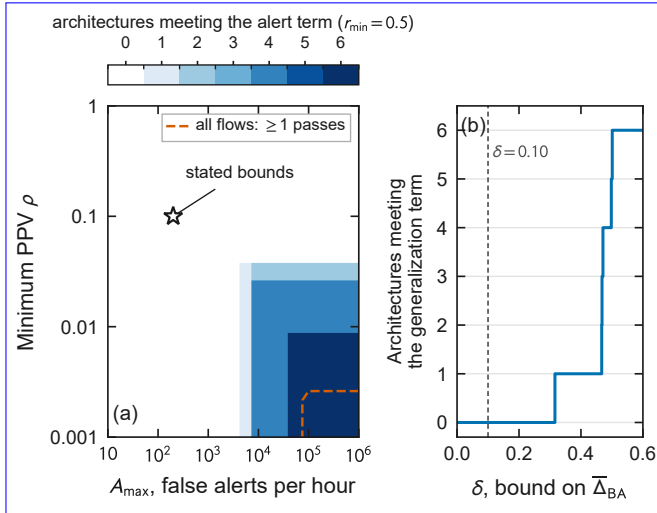


FIGURE 12. Sensitivity of the deployability test to its bounds. (a) Number of the six architectures meeting the alert term ($r_{\min} = 0.5$, $\pi = 0.002$, $\lambda_b = 240,000$ flows per hour) on $\mathcal{D}_B$ without the benign copies, as the alert bound $A_{\max}$ and the precision bound ρ vary; on all flows, at least one architecture passes only below and to the right of the dashed line. The star marks the bounds we use. (b) Number of architectures meeting the generalization term as δ varies. A detector passes the test only if both terms are satisfied.

met with a margin of about a factor of two at the smallest specified budget.

Decomposing the path shows where the time goes. Model inference accounts for to ms for the four architectures that conform at 10 ms, whereas constructing features from packets costs a median ms per flow, ranging from to ms across captures. For those architectures feature construction is between and times the cost of inference. The relation inverts for the random forest and the histogram-boosted ensemble, whose own inference exceeds extraction, so the observation is architecture-dependent and is stated as such.

Two caveats bound this. The exporter is a reference implementation in Python running at to MB/s, and **Attainability of the test.** A test that no detector passes might simply be too strict. The published 5G-NIDD pipeline described in Section VI-C indicates that this is not the case. On its own random split, with the alert term evaluated on that split, 2 of its five classifiers pass the test, with an operational precision of up to 0.79 at $\pi = 0.002$ and as few as 128 false alerts per hour at $\lambda_b = 240,000$. The same pipeline fails the test at the next rung, when the two position counters are removed, and at every subsequent rung, both on all flows and on the flows without the benign copies (Table 11). The

informativeness of the test is therefore limited by the evaluation to which it is applied. When applied to a native implementation would be substantially faster; the defensible claim is therefore that feature construction is of the same order as or larger than inference for every architecture that meets a 10 ms budget, so optimising the model alone addresses the smaller part of the budget. Per-flow cost also varies roughly eightfold across captures because it is driven by packets per flow rather than by flow count, which is why the range is reported beside the median.

random split that still contains record positions, it merely confirms the result of that split, and the pass disappears as soon as the evaluation moves one step closer to deployment conditions. The detectors examined in this paper fail the test under the evaluations closest to deployment that the available corpora permit.

VIII. COMPARATIVE ANALYSIS

A. RELATION TO REPORTED RESULTS

B. AGAINST THE REPORTED STATE OF THE ART

Direct numerical comparison with published **Most published results for O-RAN intrusion detection results are not meaningful, because those results are and 5G intrusion detection are in-distribution scores on corpora we do not share, and scores obtained on corpora that differ from ours, and our own in-distribution scores in Table ?? are in the same range.** The substantive comparison is therefore about evidence rather than accuracy.

We surveyed reporting practice using the following protocol. We queried IEEE Xplore, the ACM Digital Library and Scopus for records published between January 2021 and December 2025 matching ("O-RAN" OR "Open RAN" OR "5G") AND ("intrusion detection" OR "anomaly detection") AND ("machine learning" OR "deep learning"), restricted to peer-reviewed conference and journal papers in English. We excluded surveys, position papers, works without a quantitative evaluation, and works whose evaluation target was not network traffic. Two authors screened titles and abstracts independently and resolved disagreements by discussion, then assessed the full text of retained records against the eight reporting properties of Table ?. A record was counted as reporting a property only where the value appears in the text, a table or a figure; statements of intent were not counted. Table ? gives the resulting counts. Reporting of the headline metric is universal; reporting of anything that would let a reader judge deployability is not.

B. APPLYING THE DEPLOYABILITY PREDICATE

The three bounds in (6) require justification rather than assertion. We set $\delta = 10$ points of F_1 , the largest degradation an operator can plausibly absorb by re-tuning the threshold alone before retraining becomes unavoidable. We derive $A_{\max}$ from analyst capacity rather than choosing it: at a triage rate of twelve alerts per analyst-hour and three analysts on shift, a tier-one queue absorbs thirty-six escalations per hour, and after an automation pass the sustainable raw rate is of order two hundred per hour **scores under a random split are similarly high (Table 5). A direct numerical comparison would compare evaluation protocols, not detectors. Our results show instead which commonly reported quantities predict deployment. Held-out macro- F_1 under a random split does not. The quantities in the reporting requirements of Section X are the ones that changed our conclusions.**

$B = 10$ ms follows from the specification [?] and is not ours to choose.

Instantiating (6) with these bounds yields $\mathcal{P}(f_\theta) = 0$ for every architecture we evaluated, and each fails for a different reason. The decision tree and logistic regression satisfy the latency term and fail the generalisation and alert terms.

The random forest attains the best target score of the six and misses the latency term by more than an order of magnitude. The MLP has the best held-out source score in the

study and fails the generalisation term absolutely, transferring to below a stratified coin flip.

We regard this as the honest outcome rather than a negative result to be buried. Reporting only Table ?? under a random split would have supported a claim of a deployable high-accuracy detection xApp, and that claim would have been wrong in three independent ways.

One caveat belongs with the predicate rather than in the limitations, because it governs how (6) should be read. The third term is evaluated on *emulated* latency, and the emulation is a Python prototype rather than a deployed implementation. A published measurement inside a real Near-RT RIC reports inference three orders of magnitude faster for the same model families [23]. The latency term therefore discriminates between our implementations, not between the architectures, and it should be re-evaluated against any deployed system before it is used to exclude a candidate.

VIII. REPRODUCIBILITY AND ETHICAL CONSIDERATIONS

We release the feature-harmonisation **The code, the split manifests giving the exact device-level partition of , the five random seeds, the trained model artefacts in ONNX form, the xApp container image referenced by digest, and the replay harness used for the latency measurements. Package versions are pinned as stated in Section V, and every table in this paper is regenerated from configuration, the seeded split procedures from which every split can be regenerated deterministically, the raw measurement logs by a single script, so that the reported figures cannot drift from the underlying data. is publicly available from its authors [39]; is released under per-seed results, and the same terms as the testbed platform on which it was collected.**

The traffic analysed here originates from instrumented **generators for every table, figure, and in-text number are available at <https://github.com/adeliusa486/oran-ids-deployability>. A single script regenerates every number, table, and figure of this paper from the provided per-seed results and verifies them against the manuscript. The same script can also re-run each experiment from the raw corpora, whose files it verifies by SHA-256, and the file REPRODUCE.md in the repository describes the procedure. All three corpora are public. $\mathcal{D}_A$ is distributed by its authors under CC-BY-4.0 [41]. $\mathcal{D}_B$ is distributed by its authors under CC-BY-4.0 [39], including the release with all fields preserved and the GTP-removed captures that we used for the address audit and the re-extraction with Zeek. $\mathcal{D}_C$ is available from its authors' repository [24], which states no license, so we do not redistribute it. The RIC loop needs a Linux host or WSL2, and the repository builds FlexRIC, the xApp, and Zeek in user space without administrator rights. The traffic originates from testbed devices and a controlled test network , not from a production network carrying subscriber traffic test network and not from subscribers, so no personal data or subscriber identifiers were processed. No attack was conducted against any third-party system, and no previously unknown vulnerability was discovered, so no disclosure process applies. We note that publishing the degradation characteristics of a detector was processed. Reporting the conditions under which detectors fail carries a modest dual-use risk, in that an adversary could infer which conditions weaken it; we judge this outweighed by . We judge it smaller than the benefit to operators of knowing that published accuracy figures do not transfer , which is the premise on which such systems are currently being procured reported accuracy need not transfer to their networks.**

IX. LIMITATIONS

Several limitations qualify these findings, and the first of them bounds everything else.

The exporter confound is not controlled. Flow exporters. $\mathcal{D}_A$, $\mathcal{D}_B$, and $\mathcal{D}_C$ were produced by three different tools (Zeek, Argus, and NFStream). For $\mathcal{D}_A$ is exported by Zeek and $\mathcal{D}_B$ by Argus, and the two do not agree about where a flow ends — median flow duration differs by four orders of magnitude between them. Re-extracting both corpora with a single exporter was ruled out on cost grounds, so every Δ_{F_1} we report is an *upper bound* spanning an independently collected deployment and an independent feature-extraction pipeline. It must not be read as a measurement of deployment shift alone. The magnitude of the confound is not merely acknowledged but quantified, by others: a classifier predicting which corpus a flow came from reaches 0.993 balanced accuracy on harmonised features of these two corpora [22]., we re-extracted $\mathcal{D}_B$ with Zeek, and the gap did not close (Section VI-E). The Zeek version and configuration that produced $\mathcal{D}_A$ are not documented, so our Zeek export of $\mathcal{D}_B$ may still differ from it in timeouts or in how it handles single-packet flows. The labels of the Zeek flows come from the published labels through the host pairs of each capture. They reproduce every published label on the Argus flows, but a Zeek flow whose host pair carries both attack and benign traffic within one capture takes the attack label. We did not re-extract $\mathcal{D}_C$.

All latency figures are emulated. They were taken on a Windows host with no CPU isolation, in a single Python process, with no RIC in the path. They support relative ordering and stage decomposition, and they support *no* conformance claim. A real measurement on OpenAirInterface and FlexRIC reports inference three orders of magnitude faster for the same model families [23], which is why we attribute the near-real-time crossing point to the implementation rather than to the architecture. CPU and memory under load are not measured at all **Target labels. Of the benign flows in $\mathcal{D}_B$, 59% are exact copies of records that are labeled as UDP flood elsewhere in the corpus (Section VI-C). Their addresses identify them as the flood of the attacker at the other base station, but the dataset authors have not confirmed this, so every operating point on $\mathcal{D}_B$ is reported both with and without these flows. We recovered the capture files from the restarts of Offset. The per-station files of the release confirm the assignment of the two passes to the base stations, since each of the 10 files of the first station matches exactly one capture file of the first pass in row count and attacking host (10 of 10).**

The grouping key is confounded with attack identity, and unequally across layers. Grouping the radio layer by capture session is perfectly aligned with attack category — purity 1.000 — because each run exercises one scenario, so no radio-layer result can distinguish memorising a run from memorising an attack. Network-layer grouping by source address reaches only 20.6% of an oracle that groups directly by attack type, so the two are separable there. A run-disjoint label should not be quoted without the corresponding purity figure **Radio capture schedule. Nine of the ten benign radio sessions were recorded six weeks before the first attack session, mostly with a single UE, and every radio label is a session label. A radio detector may learn to separate capture periods in addition to attacks. The only benign session**

recorded during the attack period is flagged almost entirely, but the same holds for two early sessions (Table 7), and ten benign sessions are not sufficient to distinguish between the two explanations.

The effective sample size is smaller than it appears. Twenty split seeds resample thirty capture sessions; they do not enlarge that population. Intervals are paired t -intervals over split means rather than bootstraps for this reason, and the temporal analysis is powered to detect a large effect and nothing subtler.

Number of corpora. Three corpora are sufficient to establish that the transfer gap recurs, but not to characterize how it varies across deployments. The third corpus originates from a 5G core testbed rather than from an O-RAN deployment, contains only flood attacks and a control-plane attack, and comprises three capture files, which are too few for cluster-based intervals. No further O-RAN corpus with paired radio telemetry was available to us.

Three attack categories cannot be tested in transfer. contains no DDoS, brute-force or web attacks, so those categories are reported as untestable rather than averaged into a macro score as though they had been evaluated. **Latency.** All latency figures come from one workstation without CPU isolation. The RIC loop ran the FlexRIC software path over loopback SCTP to an emulated E2 node that reports synthetic KPM values, so it includes neither a radio, a real E2 node, nor network transport between sites. It measures the processing cost of the RIC path, not the delay of a deployment. The loop was faster at a 1 ms than at a 10 ms reporting period, which shows that its figures also depend on the power management of the host. We did not measure CPU and memory utilization under load.

Two corpora are two points. They establish that a gap exists, not how it varies across the space of deployments.

The operational prevalence $\pi = 0.002$ in (4) is a declared assumption, not a measurement. We therefore report the sweep rather than any single value, and we note that PPV **Operational parameters and the deployability test.** The prevalence $\pi = 0.002$ and the benign arrival rate λ_b are declared values and were not measured on an operational network. Both are varied over a range, and operational precision depends on π while alert volume scales linearly and independently in the benign arrival rate, so the two-dimensional parameter space collapses to one dimension plus a multiplier.

The prior correction of (??) assumes π is known, which in deployment it is not; where we apply it with a known target prior we label the result an oracle and draw no claim from it. **alone. The bounds of the deployability test** were chosen by us (Section VII-B). Although Fig. 12 reports the verdict under other bounds, an operator may weigh recall, precision, and alert volume differently, and the test does not account for detection delay or for the cost of a missed attack.

Finally, we evaluate supervised **Sample size.** The radio layer comprises 30 capture sessions, each of which contains a single scenario. Session and attack scenario are therefore confounded, and the protocol and temporal analyses can detect only large effects. Because only ten benign sessions are available, the false positive rate of the radio layer, and hence its operational precision, carries a wide interval (Section VI-G). The transfer statistics vary the training set over 20 seeds

but evaluate a single fixed target corpus, so they do not describe the variability across other deployments.

Scope. This study evaluates binary detection only. Un-supervised baselines such as isolation forests [62], feature-attribution analysis [64] and multi-class attack attribution are left to future work, and the last of these would likely transfer worse still. **Multi-class attribution, feature attribution** [64], and the fusion of radio and flow features are left for future work. The published summary files cannot be joined record by record, so fusion requires the raw archives, as in [33]. Three attack categories of $\mathcal{D}_A$ are absent from $\mathcal{D}_B$ and can be tested in one direction only.

X. CONCLUSION AND FUTURE WORK REPORTING REQUIREMENTS

An intrusion detection xApp evaluated the usual way tells you very **A held-out score on a single corpus says** little about how it will behave anywhere else. On an independently collected deployment, every architecture we tested degrades significantly, the best of them clears a stratified coin flip by of macro- F_1 **a detection xApp behaves in another deployment.** On the corpora examined here, a random split inflates the score, and a model-free lookup gains about as much, consistent with the recognition of capture sessions. The capture schedule of the radio corpus does not permit an estimate of temporal drift, and the architecture that scored *highest* on held-out source data falls *below* that floor, with a balanced accuracy of . Across six architectures there is no significant rank correlation between in-distribution and transfer performance. At the declared deployment base rate the same detectors carry corpus precision near and pooled operational precision between and , a gap of more than two orders of magnitude, while emitting tens to hundreds of thousands of mostly-false alerts per hour.

None of this is visible in the number a paper usually reports. That is the argument of this work, and the strongest evidence for it is that the argument applies to us. Two findings we previously published did not survive re-examination during this study. A reported sixfold spread in deployment precision between architectures was an artefact of averaging a non-linear functional across evaluation folds rather than pooling counts; pooled, the spread is . A methodological caveat that had been carried forward through three phases, and had determined which measurement layer the work relied on, turned out on measurement to apply more strongly to the layer it had been used to protect. We report both corrections here rather than in an erratum, because a paper arguing that evaluation practice produces misleading numbers should be judged first on its own.

false positive rate depends largely on the benign session that a detector encounters. A published accuracy of 99.9% on the target corpus depends on record-position fields, and half of that corpus lies on records that occur under both labels. The addresses of the benign copies identify them as flood traffic from a second attacker. On flows with consistent labels, detectors transferred from the source corpus achieve a balanced accuracy of 0.61 to 0.78, which is below the level attainable within the target corpus. On a third corpus, the detectors identify the SYN flood but flag a large share of the benign traffic. At a realistic base rate, no threshold yields a usable operational precision. Latency is the only criterion

the detectors meet. Through a FlexRIC loop with an emulated E2 node, a decision and its control message take at most 5.01 ms at the 99th percentile, and inference accounts for a small part of that time. We therefore propose that **papers studies** claiming a deployable O-RAN detection capability report, at minimum: (i) performance on at least one corpus not used for training or model selection, **at least the following:**

- 1) **performance on a corpus** collected independently of the training corpus; (ii) **the data, in a prevalence-invariant metric such as balanced accuracy or ROC-AUC, beside input-blind baselines and a reference trained inside that corpus;**
- 2) **the split protocol and its group key, results under a group-disjoint split, and the category coverage of any time-based split;**
- 3) **the false positive rate measured on benign-only traffic, together with on benign traffic with its unit,** the resulting alert rate per unit time, and the operational precision implied by a stated attack base rate; (iii) the 99th-percentile decision latency under a stated offered load, measured on a real RIC rather than in isolation, alongside the container CPU and memory footprint; (iv) the operating threshold actually used, with the recall it sacrifices, rather than an implicit $\tau = 0.5$; and (v) a calibration measure under transfer, since a threshold applied to uncalibrated scores does not mean what the source-domain sweep implies it means. **at a stated base rate, all from pooled confusion counts with intervals that resample clusters;**
- 4) **the operating threshold, how it was chosen, and the recall it gives up;**
- 5) **intervals that account for overlap between re-sampled splits;**
- 6) **tail latency of the loop from indication to control acknowledgment, with its stages, implementation, reporting period, and host, measured through a RIC and, before any conformance claim, with a real E2 node;**
- 7) **evidence that the labels are a function of the features used, with no identical records under opposite labels, and that no record-position or identifier field is a feature.**

We would add a sixth recommendation, which this study arrived at by getting it wrong: (vi) report operational metrics from *pooled* confusion counts, not as a mean over evaluation folds. Precision at a low base rate is strongly non-linear in the false positive rate, so a fold that happens to produce no false positives contributes a precision of exactly one. The distortion grows as the number of negatives per fold shrinks, which is precisely the regime in which the operational questions are interesting.

REFERENCES

- [1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023. doi: 10.1109/COMST.2023.3239220
- [2] A. S. Abdalla, P. S. Upadhyaya, V. K. Shah, and V. Marojevic, "Toward next generation open radio access networks: What O-RAN can and cannot do!" *IEEE Network*, vol. 36, no. 6, pp. 206–213, 2022. doi: 10.1109/MNET.108.2100659
- [3] S. Niknam, A. Roy, H. S. Dhillon, S. Singh, R. Banerji, J. H. Reed, N. Saxena, and S. Yoon, "Intelligent O-RAN for beyond 5G and 6G wireless networks," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, 2022, pp. 215–220. doi: 10.1109/GCWkshps56602.2022.10008676
- [4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016. doi: 10.1109/JIOT.2016.2579198
- [5] L. Bonati, S. D'Oro, M. Polese, S. Basagni, and T. Melodia, "Intelligence and learning in O-RAN for data-driven NextG cellular networks," *IEEE Communications Magazine*, vol. 59, no. 10, pp. 21–27, 2021. doi: 10.1109/MCOM.101.2001120
- [6] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1093–1110. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis>
- [7] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017. doi: 10.1109/MC.2017.201
- [8] G. Görbil, O. H. Abdelrahman, M. Pavloski, and E. Gelenbe, "Modeling and analysis of RRC-based signalling storms in 3G networks," *IEEE Transactions on Emerging Topics in Computing*, vol. 4, no. 1, pp. 113–127, 2016. doi: 10.1109/TETC.2015.2389662
- [9] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "CoO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing*, vol. 22, no. 10, pp. 5787–5800, 2023. doi: 10.1109/TMC.2022.3188013
- [10] R. Schmidt, M. Irazabal, and N. Nikaein, "FlexRIC: An SDK for next-generation SD-RANs," in *Proc. ACM CoNEXT*, 2021, pp. 411–425. doi: 10.1145/3485983.3494870
- [11] H. Wen, P. Porras, V. Yegneswaran, A. Gehani, and Z. Lin, "5G-Spector: An O-RAN compliant layer-3 cellular attack detection service," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2024. doi: 10.14722/ndss.2024.24527
- [12] A. Scalingi, S. D'Oro, F. Restuccia, T. Melodia, and D. Giustiniano, "Det-RAN: Data-driven cross-layer real-time attack detection in 5G open RANs," in *Proc. IEEE Conf. on Computer Communications (INFOCOM)*, 2024, pp. 41–50. doi: 10.1109/INFOCOM52122.2024.10621223
- [13] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, "5G-NIDD: A comprehensive network intrusion detection dataset generated over 5G wireless network," arXiv:2212.01298, 2022. [Online]. Available: <https://arxiv.org/abs/2212.01298>
- [14] L. Ilias, G. Doukas, V. Lamprou, C. Ntanos, and D. Askounis, "Convolutional neural networks and mixture of experts for intrusion detection in 5G networks and beyond," arXiv:2412.03483, 2024, the weighted F_1 cited is from the abstract of v2 (May 2025); a revised version appeared in *Front. Artif. Intell.*, vol. 8, art. 1708953, 2026, doi:10.3389/frai.2025.1708953. [Online]. Available: <https://arxiv.org/abs/2412.03483>
- [15] S. Layeghy and M. Portmann, "Explainable cross-domain evaluation of ML-based network intrusion detection systems," *Computers and Electrical Engineering*, vol. 108, p. 108692, 2023. doi: 10.1016/j.compeleceng.2023.108692
- [16] S. Layeghy, M. Gallagher, and M. Portmann, "Benchmarking the benchmark — comparing synthetic and real-world network IDS datasets," *Journal of Information Security and Applications*, vol. 80, p. 103689, 2024. doi: 10.1016/j.jisa.2023.103689
- [17] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," *ACM Transactions on Information and System Security*, vol. 3, no. 3, pp. 186–205, 2000. doi: 10.1145/357830.357849
- [18] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316. doi: 10.1109/SP.2010.25
- [19] ETSI, "Publicly available specification (PAS); E2 interface: General aspects and principles (O-RAN.WG3.E2GAP-R003-v04.01)," European Telecommunications Standards Institute, ETSI TS 104 038 V4.1.0, Oct. 2024. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/104000_104099/104038/04.01.00_60/ts_104038v040100p.pdf
- [20] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX*

- Security Symposium, 2022, pp. 3971–3988. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>
- [21] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, “TESSERACT: Eliminating experimental bias in malware classification across space and time,” in *Proc. 28th USENIX Security Symposium*, 2019, pp. 729–746.
 - [22] A. Abrahim and A. Edhirir, “Bidirectional cross-dataset generalization and label-efficient adaptation for 5G network intrusion detection,” *Wadi Alshatti University Journal of Pure and Applied Sciences*, vol. 4, no. 2, Aug. 2026. [Online]. Available: <https://waujpas.com/index.php/journal/article/view/509>
 - [23] L. Obiuewvi, K. J. Rechowicz, S. Jayarathna, S. H. Bouk, F. Afrin, C. N. Barati, N. Moghim, V. Nannou, M. E. Ur Rahman, and S. Shetty, “Enabling real-time AI in O-RAN: Deploying and measuring AI inside a near-RT RIC xApp,” in *Proc. 35th Int. Conf. on Computer Communications and Networks (ICCCN)*, Jul. 2026, pp. 1–9. doi: 10.1109/ICCCN69946.2026.11662852
 - [24] B. Nugraha, M. Hajizadeh, T. Niehoff, A. V. Jnanashree, T. V. Phan, D. Triantafyllopoulou, O. Krause, M. Mieth, K. Moessner, and T. Bauschert, “A comprehensive 5G dataset for control and data plane security and resource management,” in *Proc. IEEE International Conference on Cyber Security and Resilience (CSR)*, 2025, pp. 326–333. doi: 10.1109/CSR64739.2025.11130023
 - [25] L. Bonati, M. Polese, S. D’Oro, S. Basagni, and T. Melodia, “OpenRAN Gym: AI/ML development, data collection, and testing for O-RAN on PAWR platforms,” *Computer Networks*, vol. 220, p. 109502, 2023. doi: 10.1016/j.comnet.2022.109502
 - [26] L. Bonati, S. D’Oro, S. Basagni, and T. Melodia, “SCOPE: An open and softwareized prototyping platform for NextG systems,” in *Proc. ACM MobiSys*, 2021, pp. 415–426. doi: 10.1145/3458864.3466863
 - [27] N. Nikaein, M. K. Marina, S. Manickam, A. Dawson, R. Knopp, and C. Bonnet, “OpenAirInterface: A flexible platform for 5G research,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 5, pp. 33–38, 2014. doi: 10.1145/2677046.2677053
 - [28] I. Gomez-Miguel, A. Garcia-Saavedra, P. D. Sutton, P. Serrano, C. Cano, and D. J. Leith, “srsLTE: An open-source platform for LTE evolution and experimentation,” in *Proc. ACM WINTeCH*, 2016, pp. 25–32. doi: 10.1145/2980159.2980163
 - [29] M. Liyanage, A. Braeken, S. Shahabuddin, and P. Ranaweera, “Open RAN security: Challenges and opportunities,” *Journal of Network and Computer Applications*, vol. 214, p. 103621, 2023. doi: 10.1016/j.jnca.2023.103621
 - [30] D. Mimran, R. Bitton, Y. Kfir, E. Klevansky, O. Brodt, H. Lehmann, Y. Elovici, and A. Shabtai, “Security of open radio access networks,” *Computers & Security*, vol. 122, p. 102890, 2022. doi: 10.1016/j.cose.2022.102890
 - [31] ETSI, “Publicly available specification (PAS); O-RAN security threat modeling and risk assessment (O-RAN.WG11.Threat-Modeling.O-R003-v03.00),” European Telecommunications Standards Institute, ETSI TR 104 106 V3.0.0, Jun. 2025. [Online]. Available: https://www.etsi.org/deliver/etsi_tr/104100_104199/104106/03.00.00_60/tr_104106v030000p.pdf
 - [32] J. Groen, S. D’Oro, U. Demir, L. Bonati, M. Polese, T. Melodia, and K. R. Chowdhury, “Implementing and evaluating security in O-RAN: Interfaces, intelligence, and platforms,” *IEEE Network*, vol. 39, no. 1, pp. 227–234, 2025. doi: 10.1109/MNET.2024.3434419
 - [33] H. Fard, I. Komarov, and G. Wunder, “Cross-layer intrusion detection in 5G O-RAN: Gains and limits of fusing radio telemetry with network flow records,” *arXiv:2606.22450*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.22450>
 - [34] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, “Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset,” *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019. doi: 10.1016/j.future.2019.05.041
 - [35] N. Moustafa, “A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets,” *Sustainable Cities and Society*, vol. 72, p. 102994, 2021. doi: 10.1016/j.scs.2021.102994
 - [36] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, “Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning,” *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022. doi: 10.1109/ACCESS.2022.3165809
 - [37] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, “CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,” *Sensors*, vol. 23, no. 13, p. 5941, 2023. doi: 10.3390/s23135941
 - [38] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116. doi: 10.5220/0006639801080116
 - [39] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, “5G-NIDD: A comprehensive network intrusion detection dataset generated over 5G wireless network,” *IEEE DataPort*, 2022. doi: 10.21227/xtep-hv36
 - [40] Y. Siriwardhana, S. Samarakoon, P. Porambage, M. Liyanage, S.-Y. Chang, J. Kim, J. Kim, and M. Ylianttila, “Descriptor: 5G wireless network intrusion detection dataset (5G-NIDD),” *IEEE Data Descriptions*, vol. 2, pp. 358–369, 2025. doi: 10.1109/IEEEDATA.2025.3592888
 - [41] F. A. Zadeh, A. Civciss, V. Ravihansa, C. Sandeepa, and M. Liyanage, “Descriptor: 5G open radio access network multi-modal intrusion detection dataset (NetsLab-5GORN-IDD),” *IEEE Data Descriptions*, vol. 2, pp. 348–357, 2025. doi: 10.1109/IEEEDATA.2025.3614167
 - [42] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, “NetFlow datasets for machine learning-based network intrusion detection systems,” in *Big Data Technologies and Applications (BDTA/WiCON), LNCS*, vol. 371, 2021, pp. 117–135. doi: 10.1007/978-3-030-72802-1_9
 - [43] M. Sarhan, S. Layeghy, and M. Portmann, “Towards a standard feature set for network intrusion detection system datasets,” *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022. doi: 10.1007/s11036-021-01843-0
 - [44] G. Engelen, V. Rimmer, and W. Joosen, “Troubleshooting an intrusion detection dataset: the CICIDS2017 case study,” in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12. doi: 10.1109/SPW53761.2021.00009
 - [45] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, “Bad design smells in benchmark NIDS datasets,” in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2024, pp. 658–675. doi: 10.1109/EuroSP60621.2024.00042
 - [46] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, p. 100804, 2023. doi: 10.1016/j.patter.2023.100804
 - [47] G. Apruzzese, P. Laskov, E. Montes de Oca, W. Mallouli, L. Brdalo Rapa, A. V. Grammatopoulos, and F. Di Franco, “The role of machine learning in cybersecurity,” *Digital Threats: Research and Practice*, vol. 4, no. 1, pp. 1–38, 2023. doi: 10.1145/3545574
 - [48] D. Kus, E. Wagner, J. Pennekamp, K. Wolsing, I. B. Fink, M. Dahlmans, K. Wehrle, and M. Henze, “A false sense of security? Revisiting the state of machine learning-based industrial intrusion detection,” in *Proc. ACM Cyber-Physical System Security Workshop (CPSS)*, 2022, pp. 73–84. doi: 10.1145/3494107.3522773
 - [49] L. D’hooge, T. Wauters, B. Volckaert, and F. De Turck, “Inter-dataset generalization strength of supervised machine learning methods for intrusion detection,” *Journal of Information Security and Applications*, vol. 54, p. 102564, 2020. doi: 10.1016/j.jisa.2020.102564
 - [50] M. Saerens, P. Latinne, and C. Decaestecker, “Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure,” *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002. doi: 10.1162/089976602753284446
 - [51] Z. C. Lipton, Y.-X. Wang, and A. J. Smola, “Detecting and correcting for label shift with black box predictors,” in *Proc. 35th Int. Conf. on Machine Learning (ICML)*, 2018, pp. 3122–3130.
 - [52] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. on Machine Learning (ICML)*, 2017, pp. 1321–1330.
 - [53] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. V. Dillon, B. Lakshminarayanan, and J. Snoek, “Can you trust your model’s uncertainty? Evaluating predictive uncertainty under dataset shift,” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.
 - [54] T. Fawcett and A. Niculescu-Mizil, “PAV and the ROC convex hull,” *Machine Learning*, vol. 68, no. 1, pp. 97–106, 2007. doi: 10.1007/s10994-007-5011-0
 - [55] S. Rabanser, S. Günnemann, and Z. C. Lipton, “Failing loudly: An empirical study of methods for detecting dataset shift,” in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.
 - [56] B. Sun, J. Feng, and K. Saenko, “Return of frustratingly easy domain adaptation,” in *Proc. 30th AAAI Conf. on Artificial Intelligence*, 2016, pp. 2058–2065. doi: 10.1609/aaai.v30i1.10306
 - [57] C. Nadeau and Y. Bengio, “Inference for the generalization error,” *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003. doi: 10.1023/A:1024068626366

- [58] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi: 10.1023/A:1010933404324
- [59] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi: 10.1145/2939672.2939785
- [60] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [61] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734. doi: 10.3115/v1/D14-1179
- [62] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. on Data Mining (ICDM)*, 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17
- [63] X. Jin, L. E. Li, L. Vanbever, and J. Rexford, "SoftCell: Scalable and flexible cellular core network architecture," in *Proc. 9th ACM Conf. on Emerging Networking Experiments and Technologies (CoNEXT)*, 2013, pp. 163–174. doi: 10.1145/2535372.2535377
- [64] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774. [Online]. Available: <https://papers.nips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>

ARSHAD ALI is with the Faculty of Computer and Information Systems, Islamic University of Madinah, Madinah, Saudi Arabia.

ADEEL AHMAD is with the Faculty of Computer and Information Systems, Islamic University of Madinah, Madinah, Saudi Arabia.

